

Societal impacts of ML/AI

(especially fairness)



10-701 Introduction to Machine Learning
Geoff Gordon

thanks to Matt Gormley and Henry Chai for some content

Terminology for fairness in classification

- We would like to define fairness metrics for classifiers — to that end, some terminology:
- X are the non-protected features of an applicant (e.g., past offenses)
- A is a protected feature (e.g., race or gender), usually categorical, $A \in \{1, 2, \dots, C\}$ — fairness metrics compare subgroups based on A
- $Y \in \{0, 1\}$ is the true, underlying target variable, usually impossible to see or hidden until later (e.g., $Y = 1 \leftrightarrow$ this defendant would commit another crime if released)
- $h(X, A) \in \{0, 1\}$ is model's prediction, influencing some decision or action (e.g., $h(X, A) = 1 \leftrightarrow$ refuse bail)

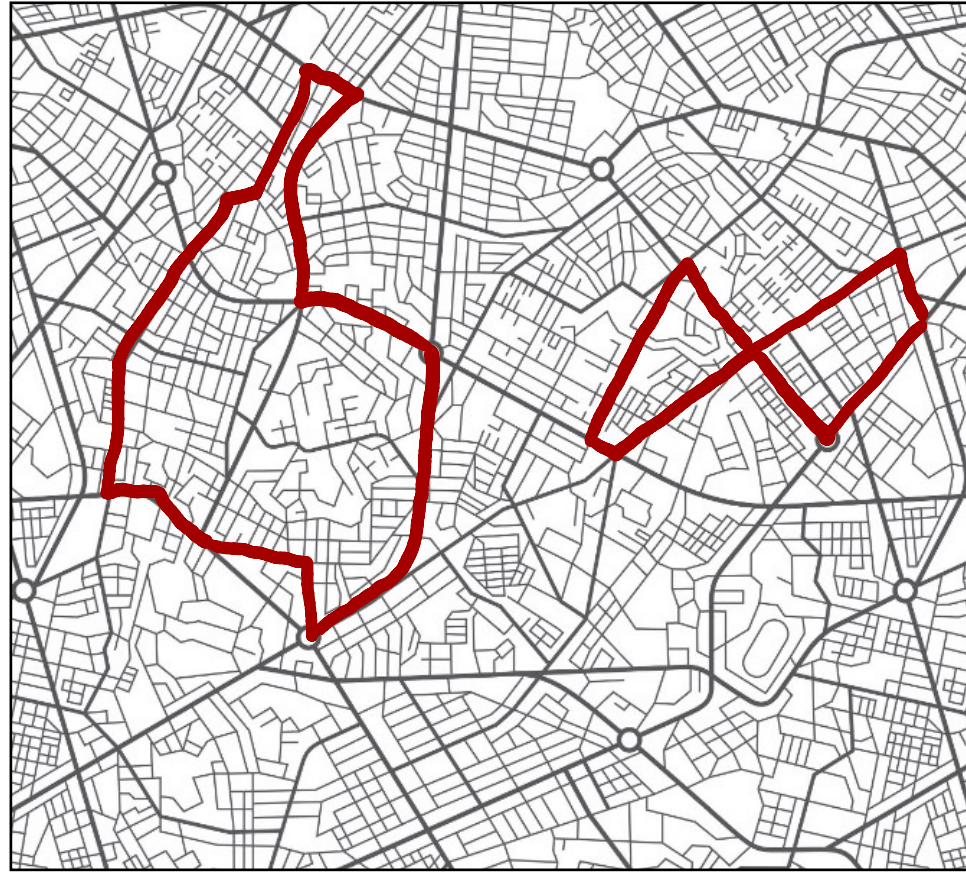
Attempt 0: Fairness through Unawareness

- Idea: build a model that only uses non-protected features *X*
- Achieves some notion of fairness:
 - ▶ “similar” individuals will receive “similar” predictions — e.g., two individuals who are identical except for their protected feature *A* would receive the same predictions

Attempt 0: Fairness through Unawareness

- Idea: build a model that only uses non-protected features X
- Achieves some notion of fairness:
 - ▶ “similar” individuals will receive “similar” predictions — e.g., two individuals who are identical except for their protected feature A would receive the same predictions
- Problem: the non-protected features X might be affected by/dependent on A
 - ▶ In general, X and A are not independent

Ex: statistical redlining



- Redlining: refusing loans based on address of applicant — end run around anti-discrimination laws
- Statistical redlining: letting a classifier learn that previous loan officers redlined — adds deniability

Healthcare risk algorithm had 'significant racial bias'

It reportedly underestimated health needs for black patients.



Jon Fingas, @jonfingas
10.26.19 in [Medicine](#)

“While it [the algorithm] didn't directly consider ethnicity, its emphasis on medical costs as bellwethers for health led to the code routinely underestimating the needs of black patients. A sicker black person would receive the same risk score as a healthier white person simply because of how much they could spend.”

Attempts 1, 2, 3: fairness metrics

- ***Independence***
- ***Separation***
- ***Sufficiency***

Independence

- A classifier satisfies ***independence*** if $h(X, A) \perp A$
 - ▶ the decision is independent of group membership (cf. attempt 0 / unawareness)
 - ▶ equivalently, ***selection rate*** is the same for all groups:
 $P(h(X, A) = 1 \mid A = a) = p \quad \forall a$
 - ▶ or ***rejection rate*** is the same for all groups:
 $P(h(X, A) = 0 \mid A = a) = 1 - p \quad \forall a$

Achieving independence

- Could edit labels, delete/replicate examples, or reweight examples so that $Y \perp A$ in training data
- Idea: if training set satisfies independence, we can hope our learner will at least approximately maintain it
 - ▶ remember the monitoring stage of ML pipeline

Achieving independence

- Or, add a penalty: writing $MI(Y, H_w)$ for train set estimate of mutual information between Y and $H_w = h_w(X, A)$,

$$\min_w \sum_t L(Y^{(t)}, h_w(X^{(t)}, A^{(t)})) + \lambda MI(Y, H_w)$$

- Gradient of MI gives a penalty for predicting positive in a group that has too many positives, or for predicting negative in a group that has too many negatives
 - ▶ could equivalently add such penalties directly
- Tries to get independence on *training set* — noise or distribution shift could affect test-time independence
 - ▶ monitoring!
 - ▶ same worry for next methods

Achieving independence

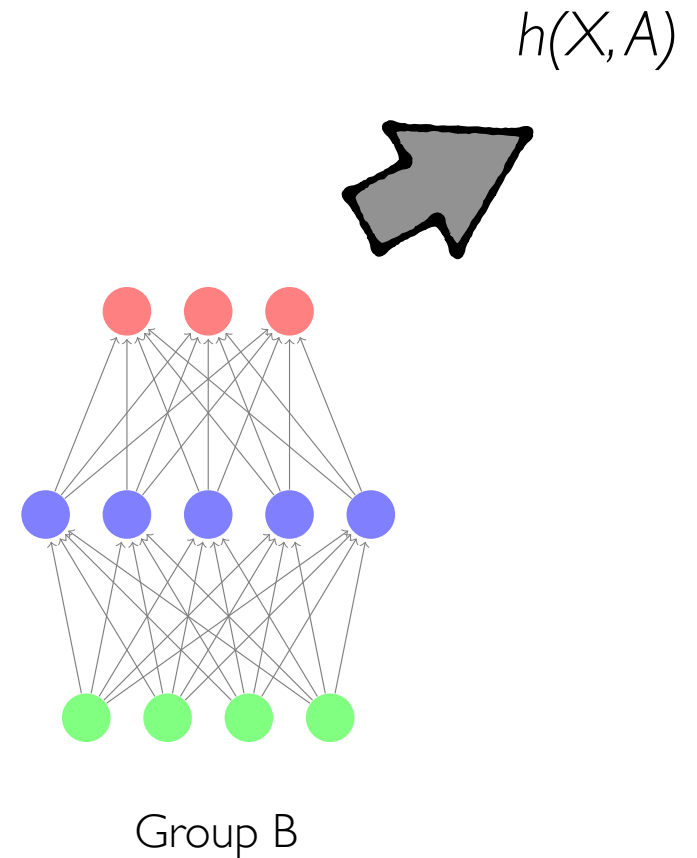
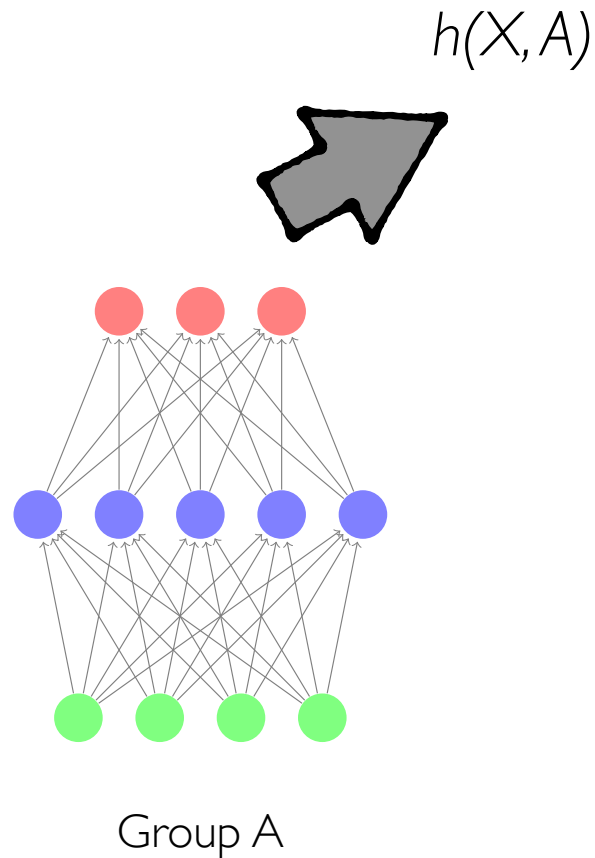
- Could post-process the classifier
 - ▶ many classifiers work by testing $\text{score}_w(X, A) \geq \tau$
 - ▶ use different thresholds τ_a for different subgroups
 - ▶ e.g., grant bail to lowest-risk X% in each subgroup

Achieving independence

- Could post-process the classifier
 - ▶ many classifiers work by testing $\text{score}_w(X, A) \geq \tau$
 - ▶ use different thresholds τ_a for different subgroups
 - ▶ e.g., grant bail to lowest-risk X% in each subgroup
- Problem: could conflict with procedural fairness
 - ▶ enforcing a quota could treat defendants of different groups unequally
 - ▶ but it doesn't have to mean this: if the classifier is biased to start, enforcing quota could *improve* procedural fairness
- Quotas can also interact w/ other fairness concepts, e.g., restorative justice
- A recurring theme: different definitions of fairness can interact or conflict

Independence via invariant prediction

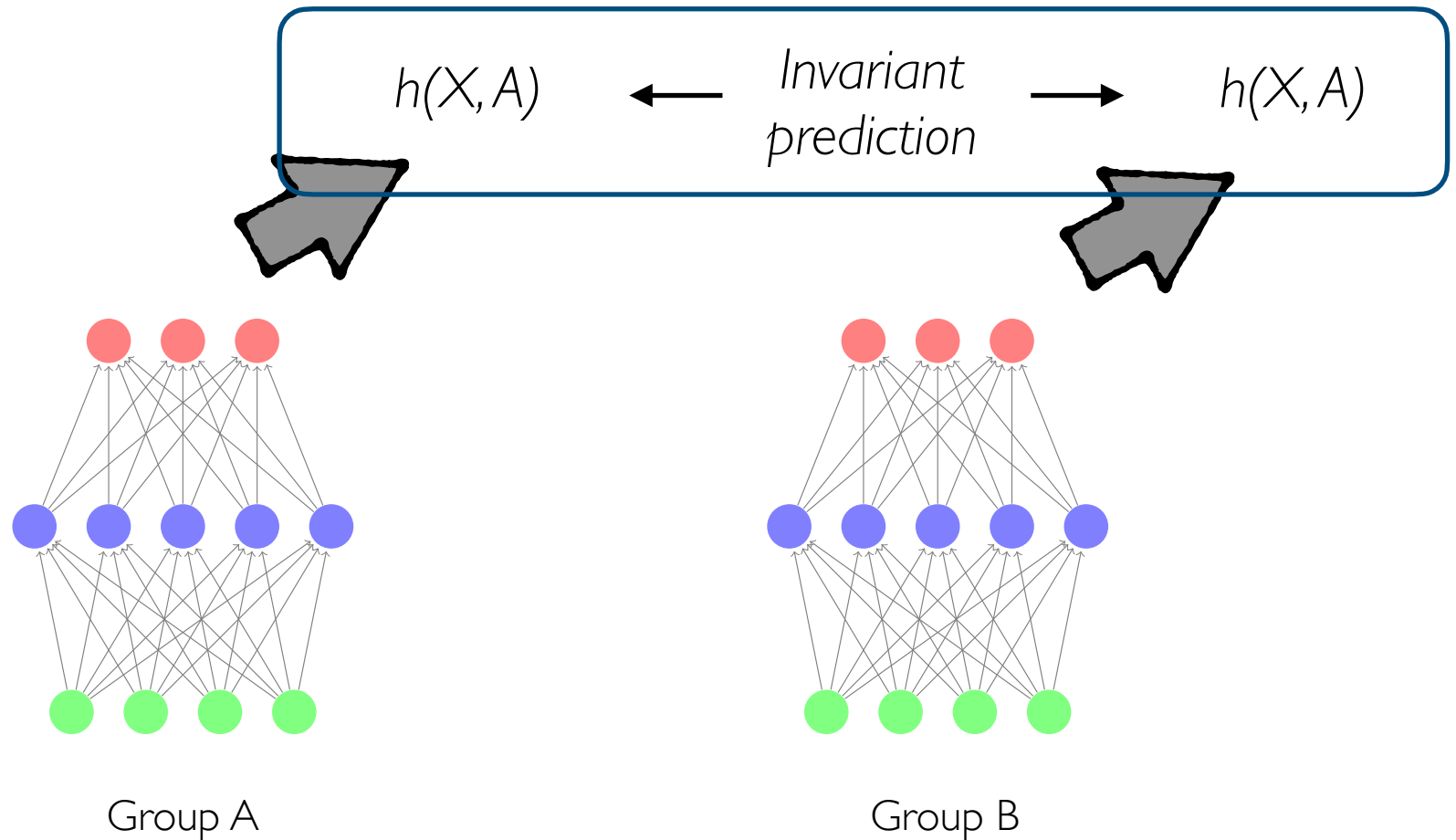
Invariant = distribution
doesn't change across groups



- Constrain prediction during training:
 - ▶ $P(\text{recommendation} \mid \text{group A}) = P(\text{recommendation} \mid \text{group B})$

Independence via invariant prediction

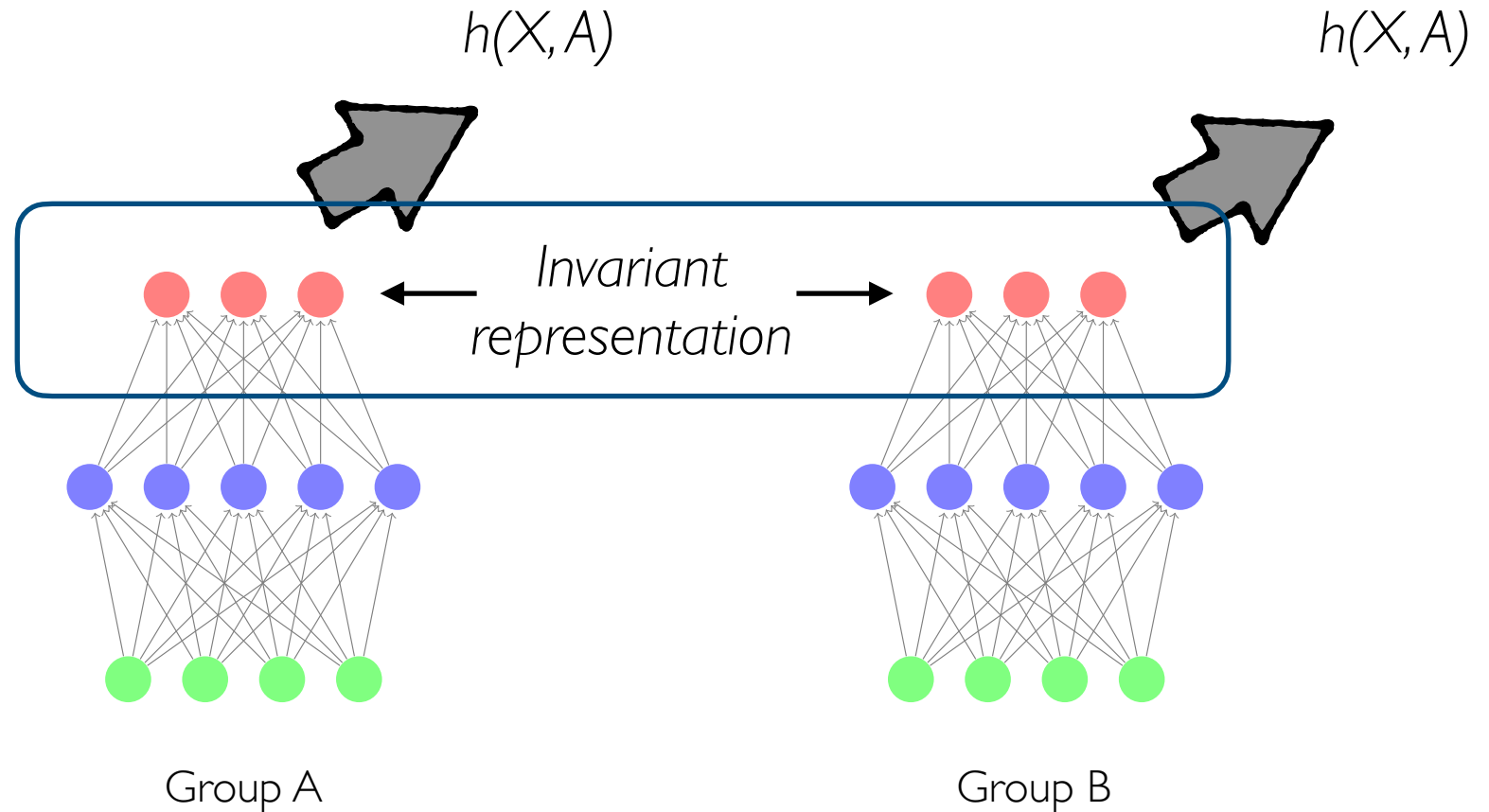
Invariant = distribution
doesn't change across groups



- Constrain prediction during training:
 - ▶ $P(\text{recommendation} \mid \text{group A}) = P(\text{recommendation} \mid \text{group B})$

Independence via invariant representation

Invariant = distribution doesn't change across groups



- Constrain $P(\text{last layer} \mid \text{group A}) = P(\text{last layer} \mid \text{group B})$
 - ▶ harder to implement: high dimensional
 - ▶ but more directly enforces that the decision process is independent of group membership

Adversarial learner

- Can enforce independent representation via an adversary
 - ▶ a **discriminator** model that tries to predict whether representations came from group A or group B
 - ▶ if $P(\text{group} \mid \text{representation}) \neq P(\text{group})$, adversary can win (group is not $\perp$ representation, equivalent to def'n above)
- Soft constraint: $\min_w \max_\phi \underbrace{L(w)}_{\text{original loss}} + \lambda \underbrace{D(\phi)}_{\text{discriminator performance}}$
- Harder to optimize, but (e.g.) both players can run SGD with strongly different learning rates

Accuracy

- All above methods can ***reduce accuracy*** on training set
- Do we care?

Accuracy

- All above methods can **reduce accuracy** on training set
- Do we care?
 - ▶ maybe: remember our goal is not training set accuracy
 - ▶ training set labels could be biased — if so, must reduce accuracy to increase actual performance
 - ▶ classifier can overfit the training set — restrictions can increase actual performance via regularization
 - ▶ most we can say: enforcing independence (or any other fairness criterion) **might** reduce accuracy; should probably monitor after deployment
- Some sources frame a tradeoff between accuracy and fairness, but this framing is misleading-to-outright-wrong
 - ▶ it could be worth reducing accuracy to increase fairness, but we shouldn't assume that decrease in training set accuracy = decrease in actual accuracy

Flaws with independence

- Can behave unpredictably if label and group are strongly dependent in training data
 - ▶ e.g., data from strongly biased labelers, or very non-comparable populations in different groups
 - ▶ best fix in this case is to improve task definition or data collection
- An adversary can still subvert the overall process while maintaining independence
 - ▶ e.g, accept X% of one group according to classifier score, and X% of other group at random (*lazy* treatment of group)
 - ▶ this is a possible outcome even without adversarial intent, if our data collection process has problems: e.g., very small or non-representative sample from one group

Attempts 1, 2, 3: fairness metrics

- ***Independence*** (selection rate parity): $h(X, A) \perp A$
 - ▶ Proportion of accepted applicants is the same for all groups
 - ▶ Flaws such as lazy behavior
- ***Separation***
- ***Sufficiency***

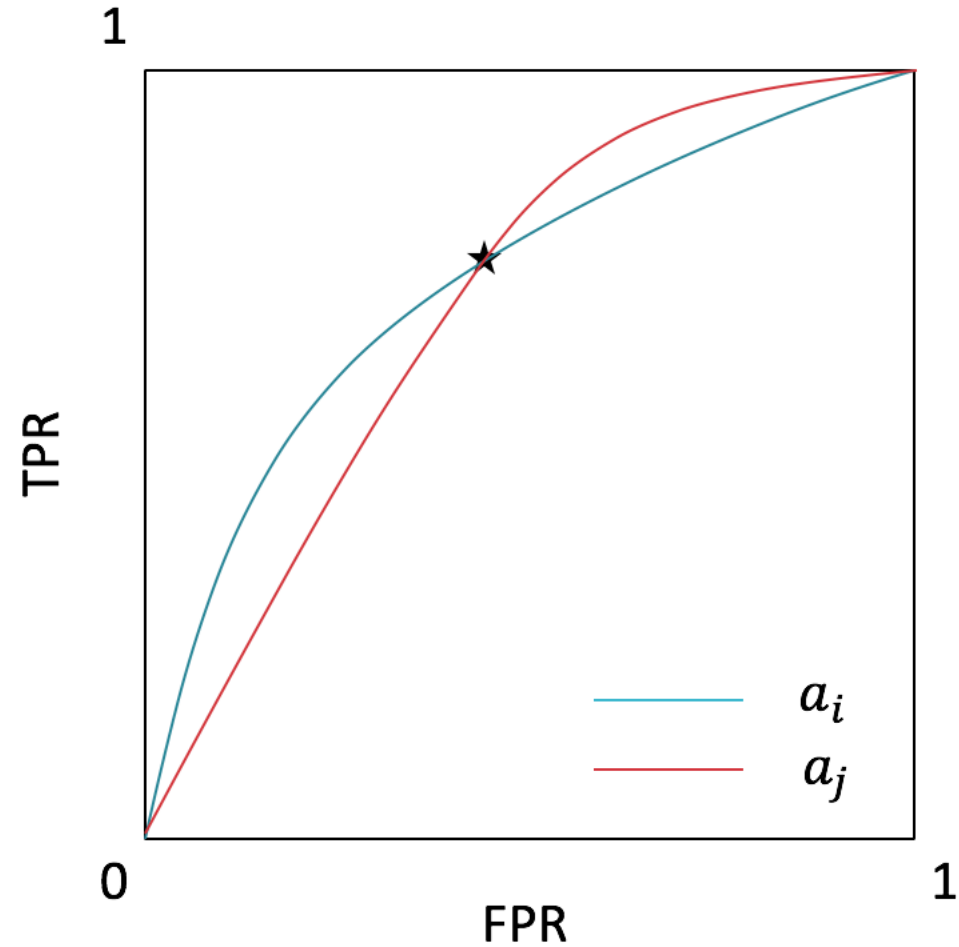
Separation

The COMPAS article
talked about violation
of separation

- A classifier satisfies **separation** if $h(X, A) \perp A \mid Y$
 - ▶ cf. independence: $h(X, A) \perp A$
 - ▶ dependence is allowed between prediction and group membership, **if** it can be explained by true labels
$$\begin{aligned} P(h(X, A) = 1 \mid A = a, Y = 1) &= p & \forall a \\ P(h(X, A) = 1 \mid A = a, Y = 0) &= q & \forall a \end{aligned}$$
- Equivalent to equality of FPR and FNR across groups
 - ▶ balance the kinds of mistakes we make
- Can be more robust than independence if training data has different proportions of labels in different groups
- Flaw: if labels are biased, this bias is perpetuated — classifier is incentivized to incorporate it

Achieving separation

Definition: ROC curve

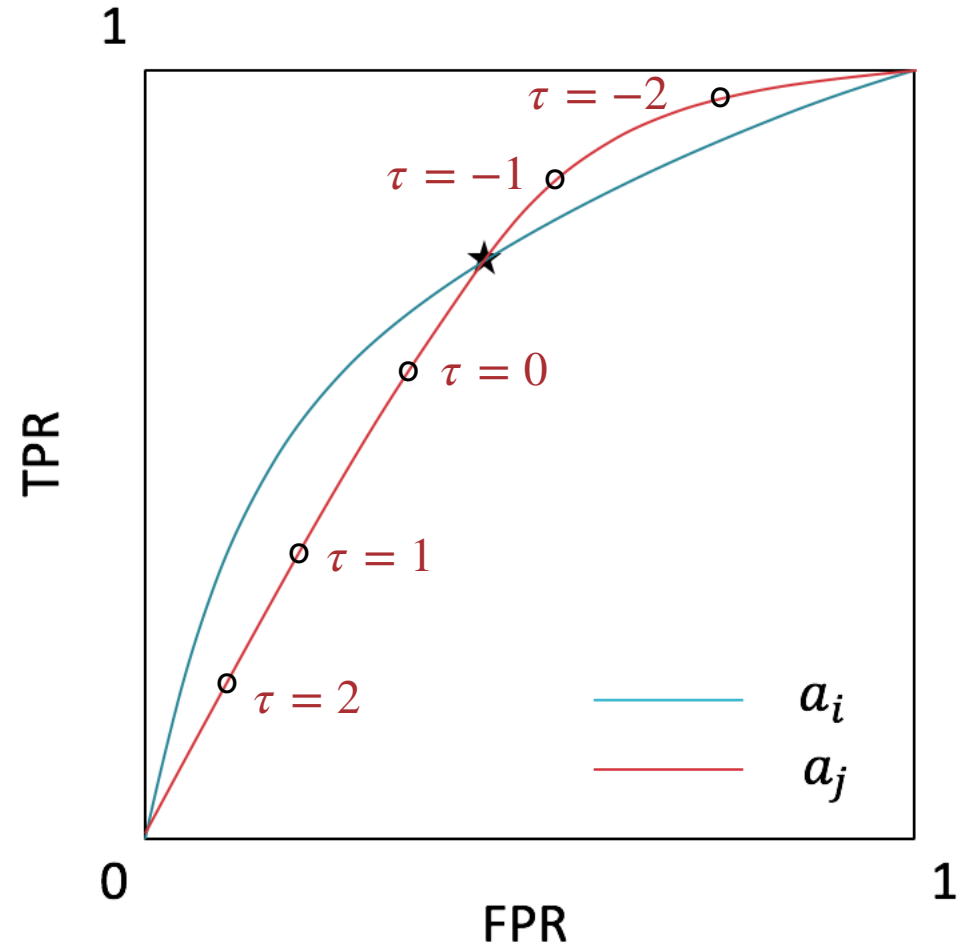


For a score-based classifier, ROC curve plots $\text{TPR} = 1 - \text{FNR}$ against FPR at different thresholds τ

$$h(X, A) = \mathbb{I}(\text{score}_w(X, A) \geq \tau)$$

Achieving separation

Definition: ROC curve

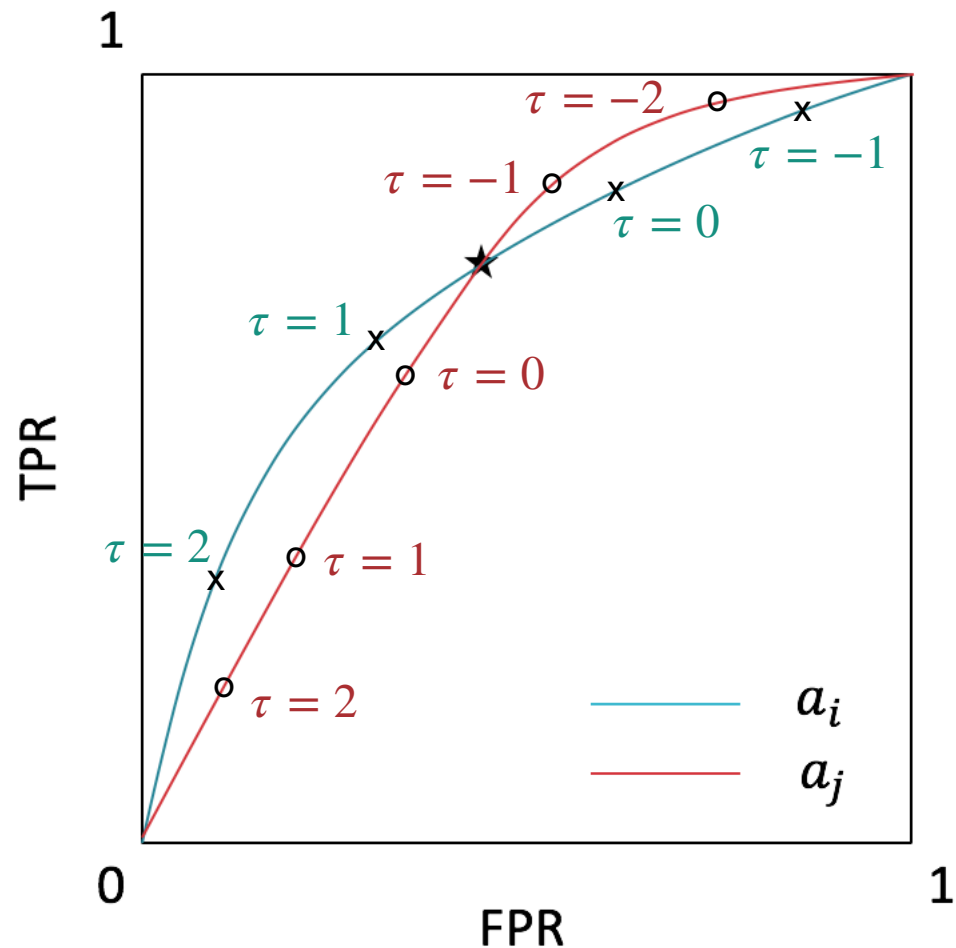


For a score-based classifier, ROC curve plots $\text{TPR} = 1 - \text{FNR}$ against FPR at different thresholds τ

$$h(X, A) = \mathbb{I}(\text{score}_w(X, A) \geq \tau)$$

Achieving separation

Definition: ROC curve

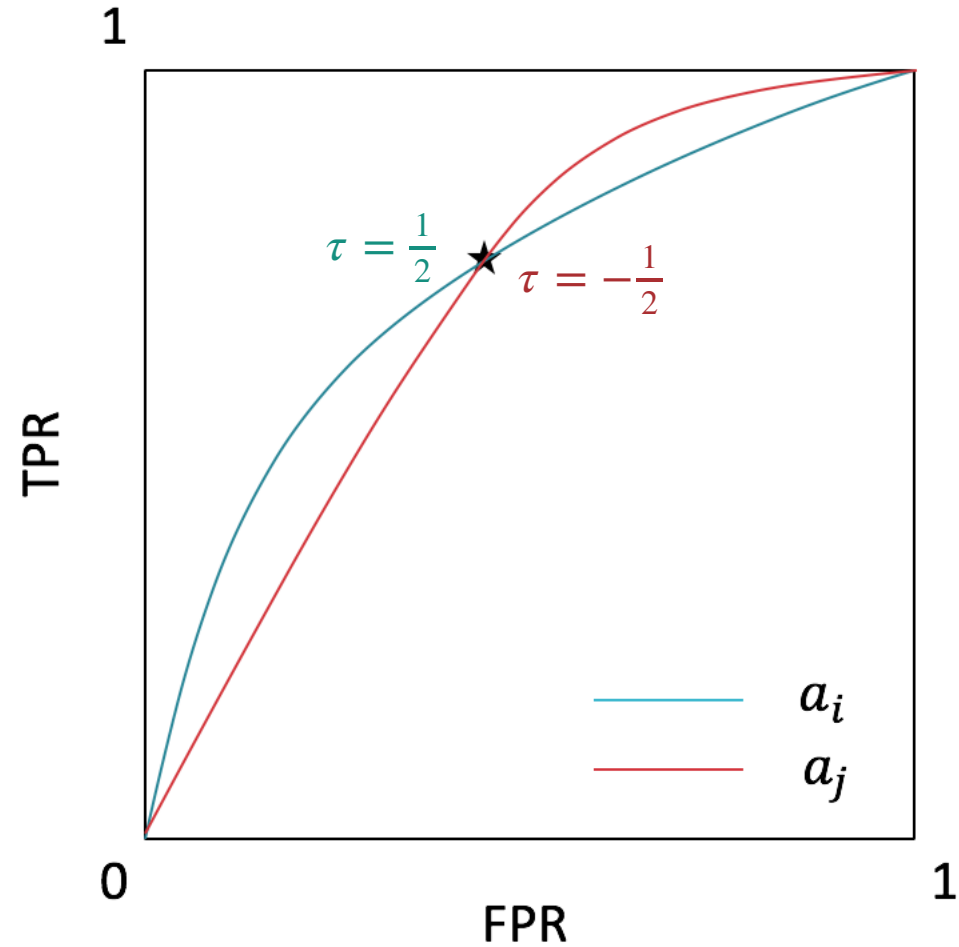


For a score-based classifier, ROC curve plots $\text{TPR} = 1 - \text{FNR}$ against FPR at different thresholds τ

$$h(X, A) = \mathbb{I}(\text{score}_w(X, A) \geq \tau)$$

Achieving separation

Definition: ROC curve

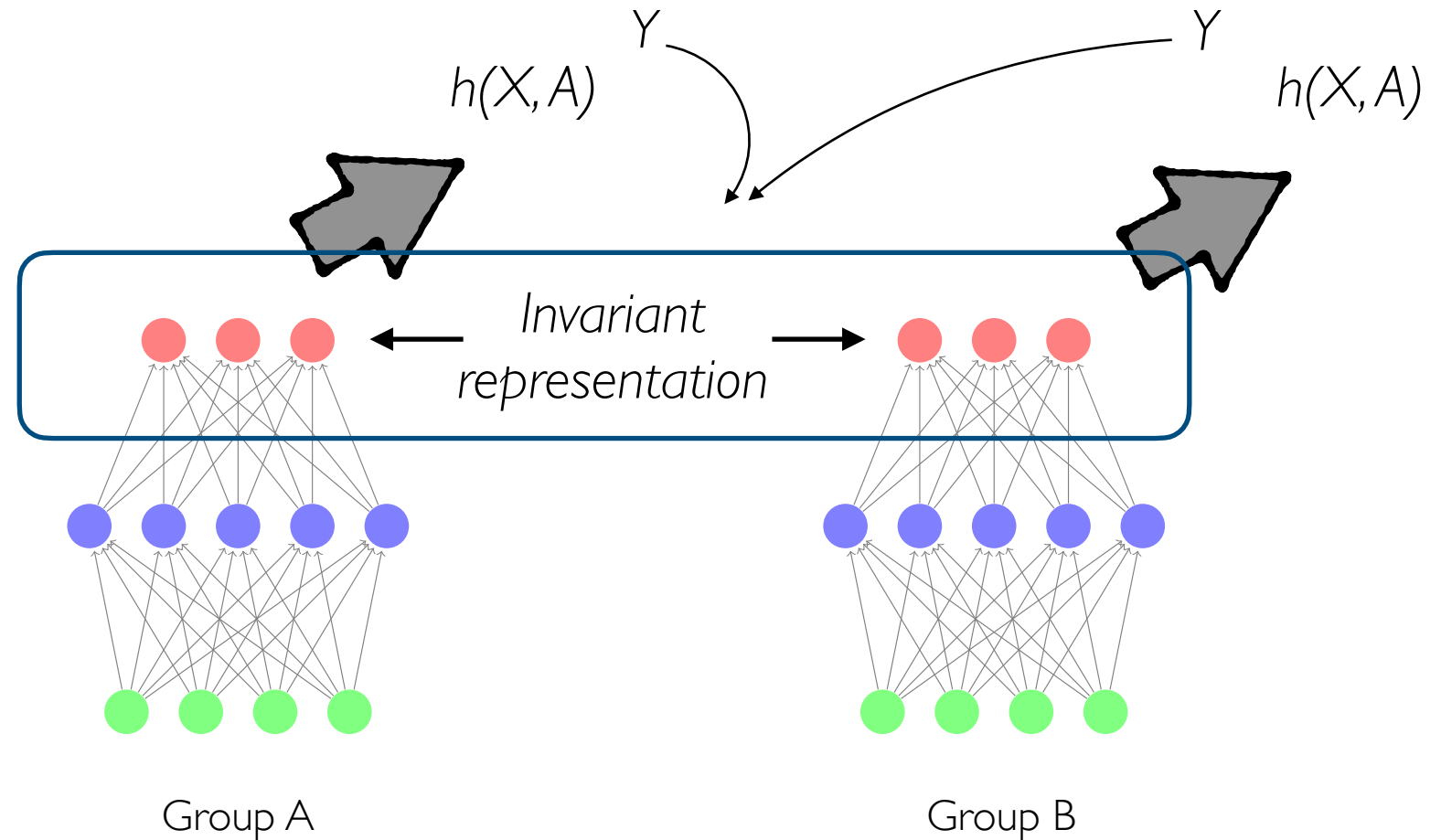


Can achieve separation by using different thresholds for different groups, corresponding to where their ROC curves intersect

if they do

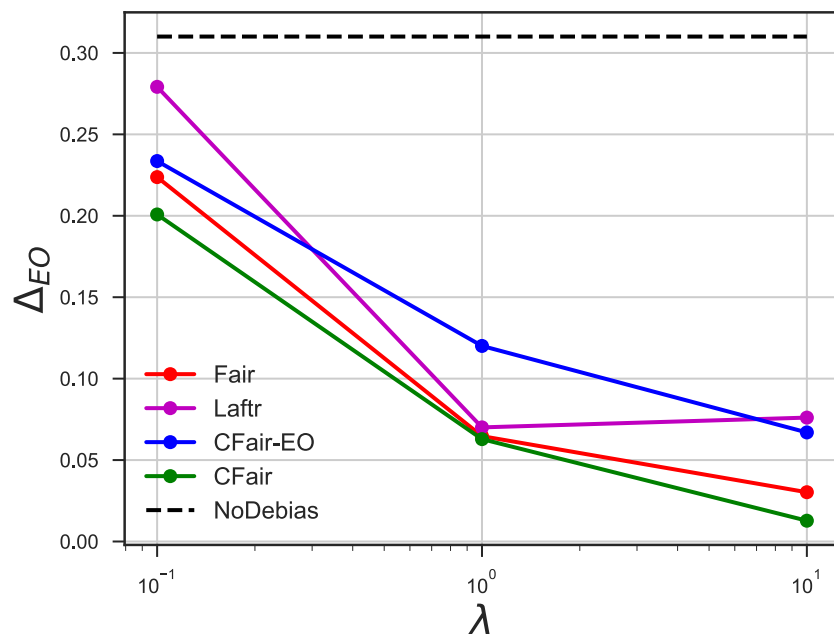
$$h(X, A) = \mathbb{I}(\text{score}_w(X, A) \geq \tau)$$

Separation via invariant representation

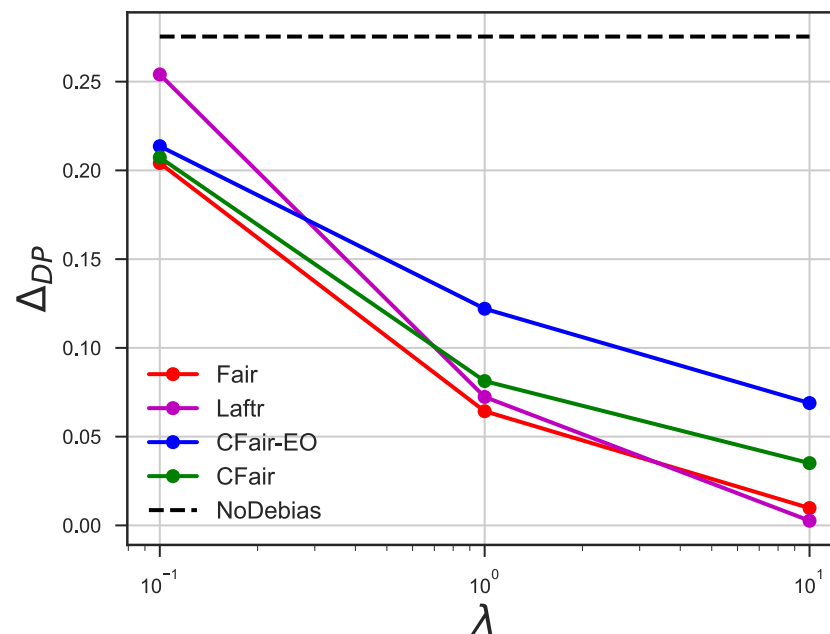


- Constrain $P(\text{last layer} \mid \text{group A}, Y) = P(\text{last layer} \mid \text{group B}, Y)$
 - ▶ last layer $\perp$ group $\mid Y$, so prediction $\perp$ group $\mid Y$
- Discriminator sees both last layer and true label, predicts group

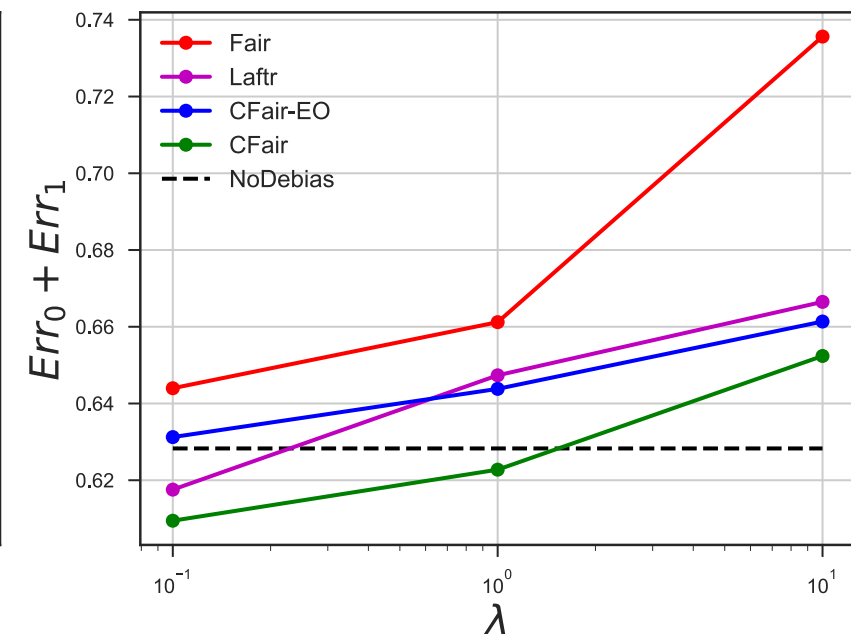
observed separation (equality of odds)



observed independence (demographic parity)



observed (balanced) error rate



Experiment

- Compare observed separation, independence, and error rate for different soft constraints (strength λ)
 - ▶ — — no constraint; — independence; — separation
 - ▶ constraints enforced on *invariant representation*
- Separation seems to achieve a good tradeoff

[Zhao et al., ICLR 2020]

Attempts 1, 2, 3: fairness metrics

- ***Independence*** (selection rate parity): $h(X, A) \perp A$
 - ▶ Proportion of accepted applicants is the same for all groups
 - ▶ Flaws such as lazy behavior
- ***Separation*** (equality of FPR and FNR): $h(X, A) \perp A \mid Y$
 - ▶ Balance the kinds of mistakes we make across groups
 - ▶ Perpetuates existing biases in the training data
- ***Sufficiency***

Sufficiency

- A classifier is **sufficient** if its prediction renders the group and label independent: $Y \perp A \mid h(X, A)$
 - ▶ $P(Y = 1 \mid A = a, h(X, A) = 1) = p \quad \forall a$
 $P(Y = 1 \mid A = a, h(X, A) = 0) = q \quad \forall a$
- Equivalent to equality of PPV and NPV across groups

Calibration

It turns out COMPAS was trained to satisfy calibration instead of separation

It's impossible in general to guarantee both — another conflict between fairness concepts

- A score-based classifier is **calibrated** if $P(Y = 1 \mid \text{score}_w(X, A) = \tau) = \tau$
 - ▶ i.e., if its score is an accurate representation of the probability that $Y = 1$
 - ▶ for real-valued scores we often post-process: pick a monotone function $\mathbb{R} \rightarrow [0, 1]$ so that the above is true
- If a classifier is calibrated *within each group*, $P(Y = 1 \mid \text{score}_w(X, A) = \tau, A = a) = \tau \quad \forall a$, then it is sufficient
- There are many procedures designed to calibrate a score-based classifier, so it's typically easy to ensure calibration (overall and within each group)
- But beware! Calibration on biased data means mimicking the bias

Attempts 1, 2, 3: fairness metrics

- **Independence** (selection rate parity): $h(X, A) \perp A$
 - ▶ Proportion of accepted applicants is the same for all groups
 - ▶ Flaws such as lazy behavior
- **Separation** (equality of FPR and FNR): $h(X, A) \perp A \mid Y$
 - ▶ Balance the kinds of mistakes we make across groups
 - ▶ Perpetuates existing biases in the training data
- **Sufficiency** (equality of PPV and NPV): $Y \perp A \mid h(X, A)$
 - ▶ For the purposes of predicting Y , information contained in $h(X, A)$ is “sufficient”, A becomes irrelevant

Attempts 1, 2, 3: fairness metrics

- **Independence** (selection rate parity): $h(X, A) \perp A$
 - ▶ Proportion of accepted applicants is the same across all groups
 - ▶ Flaws such as lazy behavior
- **Separation** (equality of FPR and NPV): $h(X, A) \perp A \mid Y$
 - ▶ Balance the kinds of mistakes made across groups
 - ▶ Perpetuates existing bias in the training data
- **Sufficiency** (equality of PPV and NPV): $Y \perp A \mid h(X, A)$
 - ▶ For the purpose of predicting Y , information contained in $h(X, A)$ is sufficient, A becomes irrelevant

Any pair of these conditions is mutually exclusive in almost all situations!

So what do we do?

- There is no mechanical way to achieve fairness
- Have to look at the system level
 - ▶ discover sources of unfairness (biased labeling, non-representative training data, consequences of system-level issues) and work to understand them
 - ▶ monitor effects of our choices
- It may be clear how to correct some problems (e.g., collect better data)
- Others will be harder: require negotiation among stakeholders, long-term coordinated action

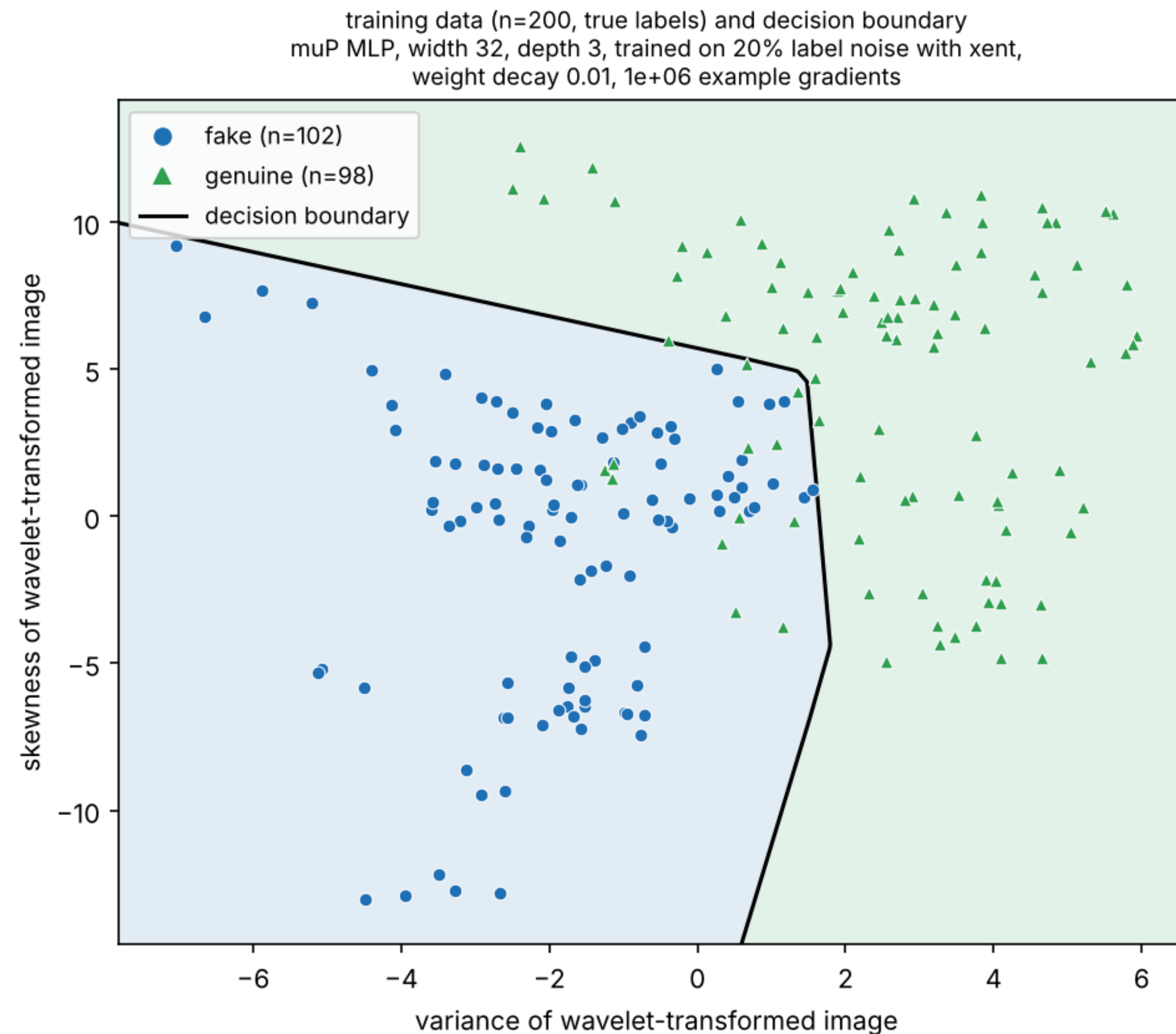
Learning objectives — fairness

- High-profile cases of unfairness and unintended consequences are increasingly common as ML/AI are applied in a broad variety of contexts
- To prevent or fix, need to look at entire system and entire training pipeline, and involve all stakeholders
- Various definitions of fairness of just the inner classifier
 - ▶ Selection rate parity (Independence): $h(X, A) \perp A$
 - ▶ Equality of FPR and FNR (Separation): $h(X, A) \perp A \mid Y$
 - ▶ Equality of PPV and NPV (Sufficiency): $Y \perp A \mid h(X, A)$
 - ▶ In all but the simplest of cases, any two of these three are mutually exclusive
- And even more definitions at system level! E.g., allocative, participatory, or procedural fairness
 - ▶ no definition gives whole picture, conflicts are common

Overfitting and deep nets

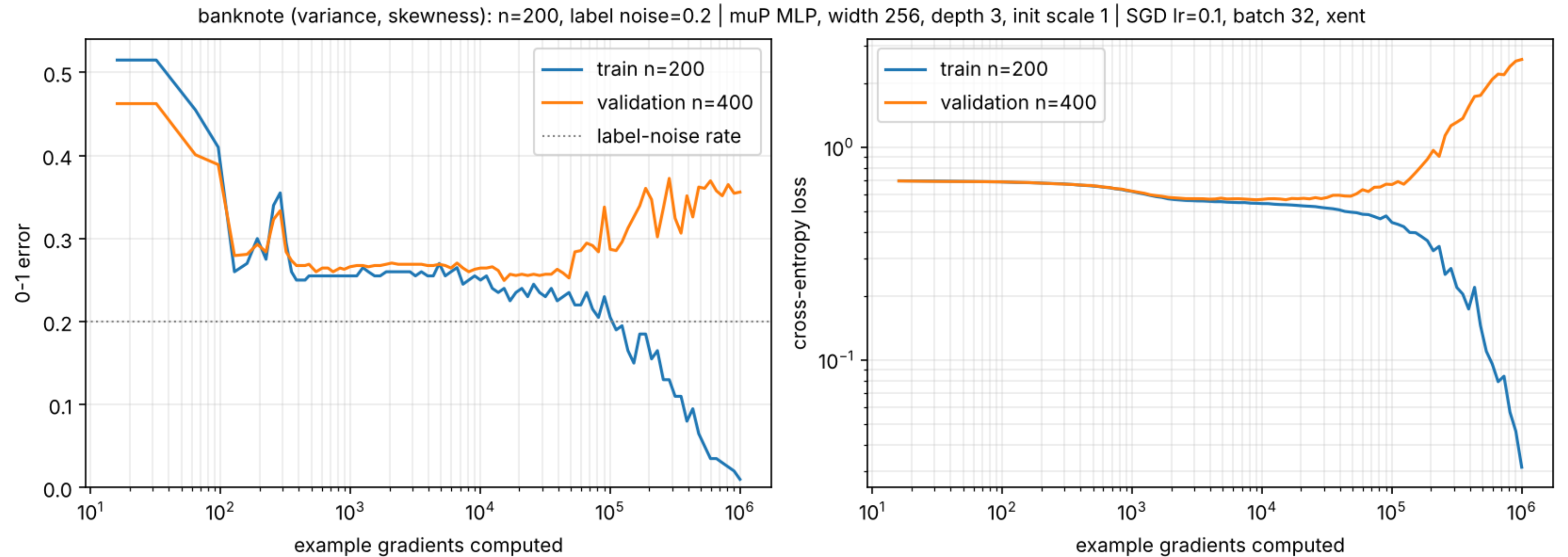
- Overfitting in deep nets behaves very differently from overfitting in classical models
 - ▶ more like second phase of double descent, since we can typically interpolate training set, but not a perfect match
- Recall two effects of overfitting
 - ▶ difference between training and validation/test error (optimism)
 - ▶ difference between best representable model and actual trained model (excess risk)
- Deep nets often have first effect, but it isn't necessarily harmful
- If we aren't careful, second effect (excess risk) can be large — we find a bad model

Data for overfitting example



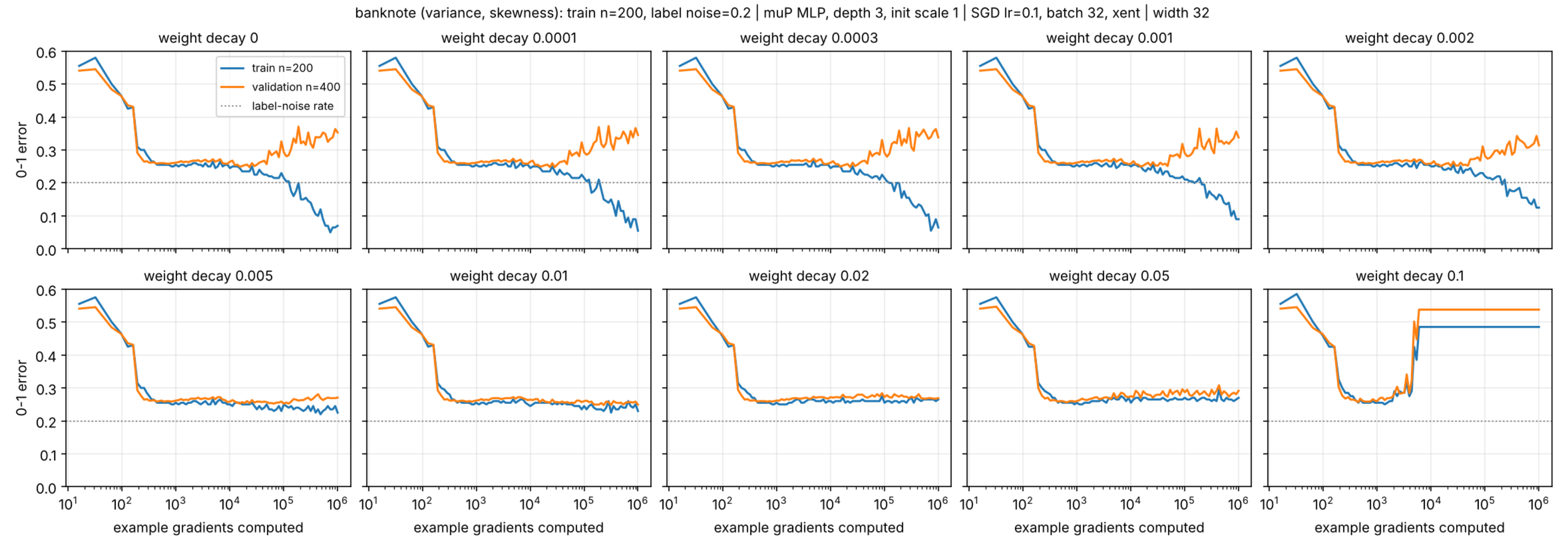
- Recall the UCI banknote data
 - ▶ using 2 of 4 features, picked this data because it has a nonlinear decision boundary
 - ▶ we'll flip 20% of labels since this increases overfitting

Early stopping



- Surprising effect of SGD: limiting the number of iterations is a good regularizer — **early stopping**
 - ▶ we have partial explanations (limits weight norm, largest eigenvalue of Hessian) but none are perfect
 - ▶ effect varies by optimizer; the most-used ones (e.g., SGD, Adam) are popular partly because they early-stop well

Weight decay can limit overfitting

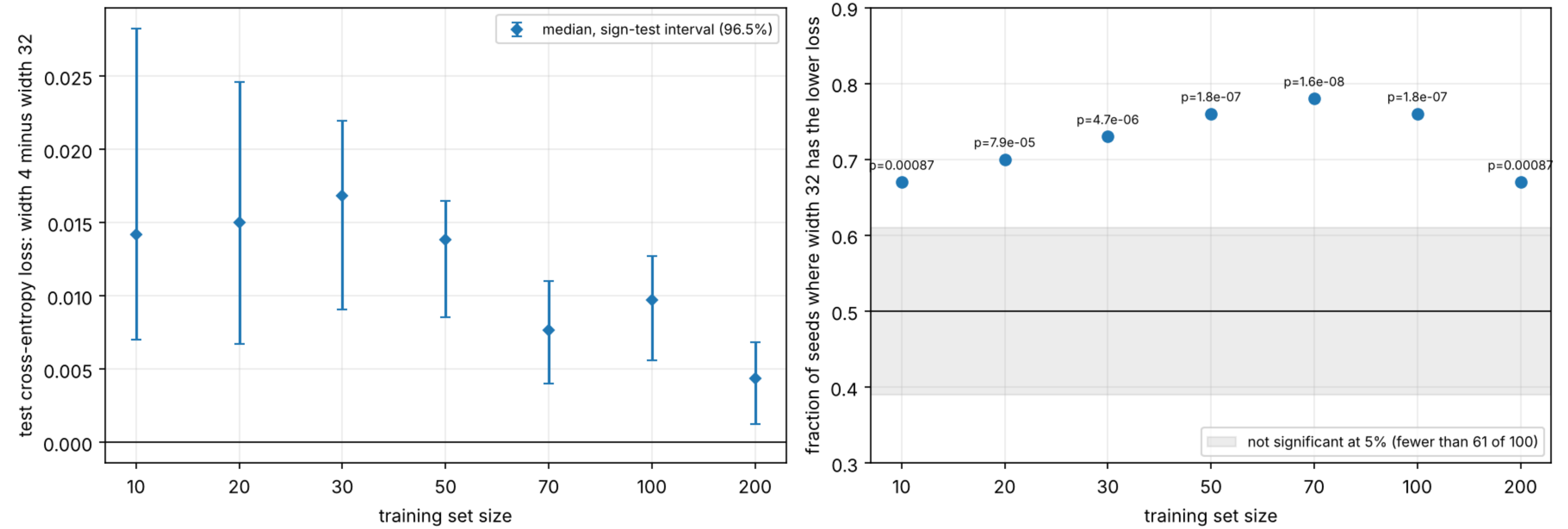


- Decay parameter needs proper tuning (and scaling with network size) but can take the place of early stopping
 - ▶ too little: still get overfitting; too much: constant predictor
- Even better: mix of weight decay and early stopping

Model choice

*Here: wider model has slightly lower best-case error, is substantially **less** susceptible to overfitting (also to optimizer failure, not shown)*

xent test error difference: (width-4 – width-32)



- Even after best practices (well chosen weight decay and early stopping, good initialization and learning rates), different model classes can overfit more or less
 - ▶ unlike classical ML, known measures of class complexity (size, VC dimension, parameters) are basically irrelevant
 - ▶ but still $\exists$ differences among classes, and it's good to try to understand by experiment (and theory when possible)

High-level advice

- Best practices to reduce excess risk from overfitting
 - ▶ use early stopping and/or weight decay — without these, serious overfitting problems
 - ▶ explore differences among model classes
- Choosing a model class
 - ▶ make sure model class is rich enough to easily overfit the training data (in the first sense, optimism)
 - ▶ e.g., increase width and/or depth
 - ▶ depth can make optimizer harder to tune, width typically helps optimizer — main costs are compute and memory
 - ▶ can't compare classes before we tune decay/stopping
 - ▶ combination of expressive class with well-tuned decay/stopping is typically best choice overall

Convolutional networks



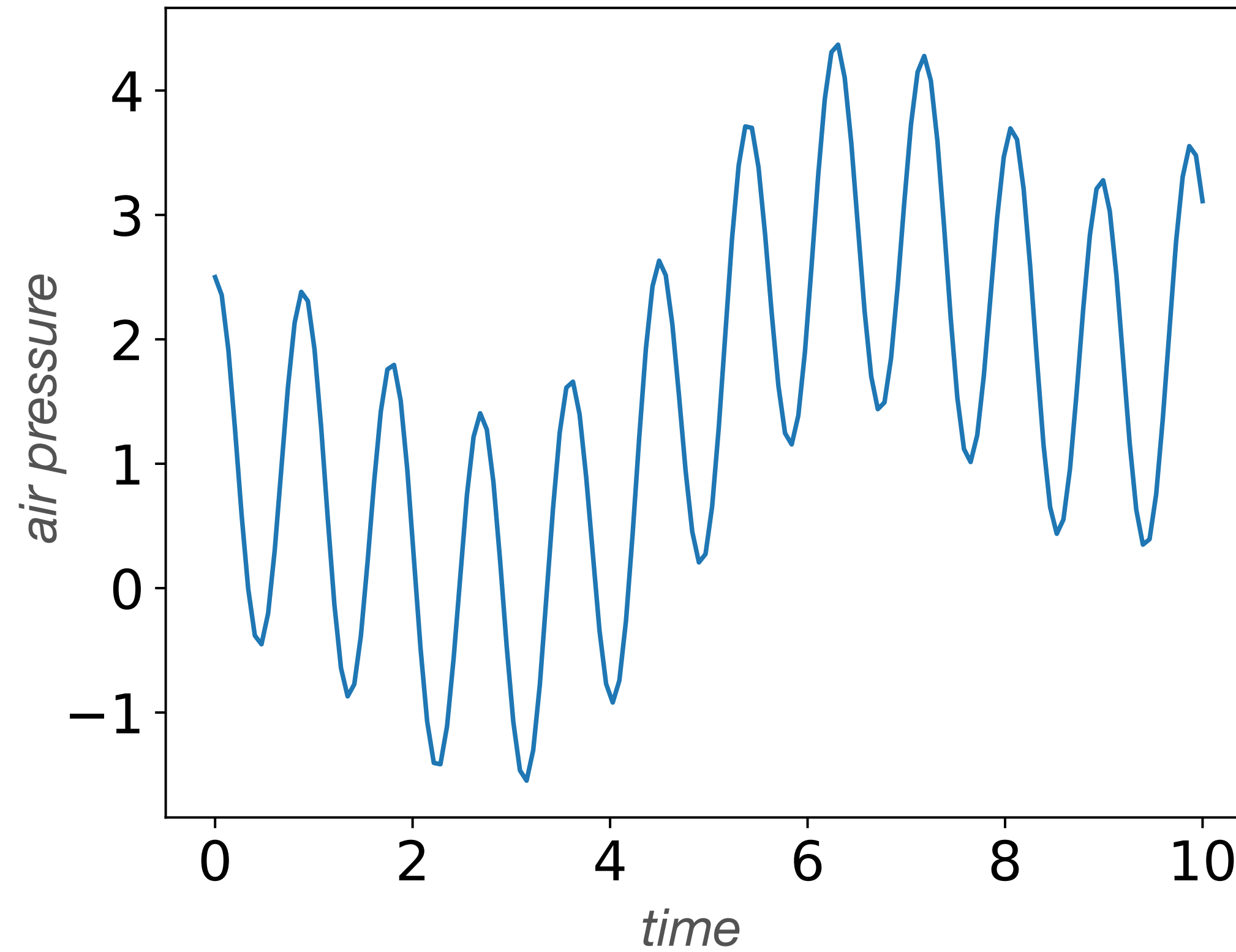
10-701 Introduction to Machine Learning

Geoff Gordon

High-dimensional learning

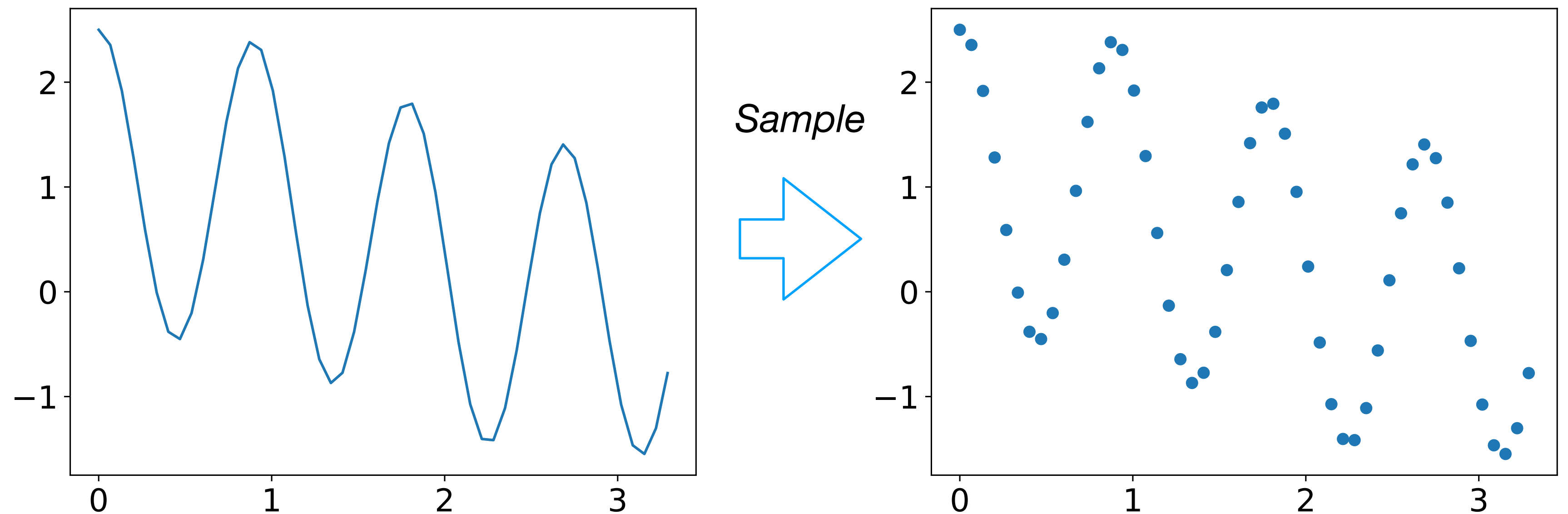
- In ML we often need to learn predictors for high-dimensional inputs and outputs
 - ▶ means we need to work with high-dimensional tensors (algebra, differentiation)
- Example: ***signal processing*** or prediction from audio, images, video, ...

Signal



- Signal: function from $\mathbb{R}^d \rightarrow \mathbb{R}$
- E.g., sound waves
 - ▶ here $d = 1$ represents time

Sampled signal



- Continuous-time signal $\xrightarrow{\text{sampling}}$ vector in $\mathbb{R}^k$
 - ▶ instead of $f(0.134) = 1.92$, we have $x_3 = 1.92$
- We do this every time we make a graph of a function
 - ▶ think of f as an infinite-d vector, x as its finite-d version
 - ▶ argument t in $f(t) \sim$ index i in x_i

Signal processing

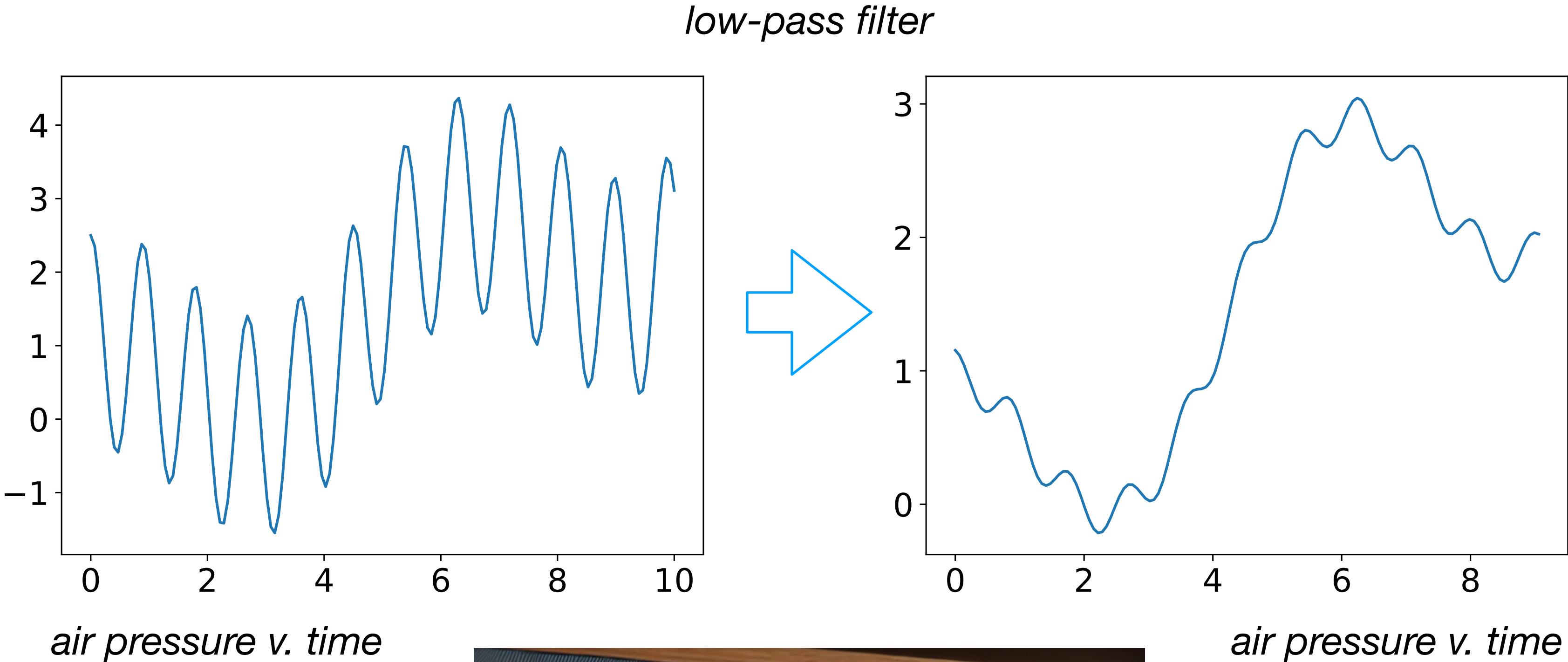


image credit: ChatGPT

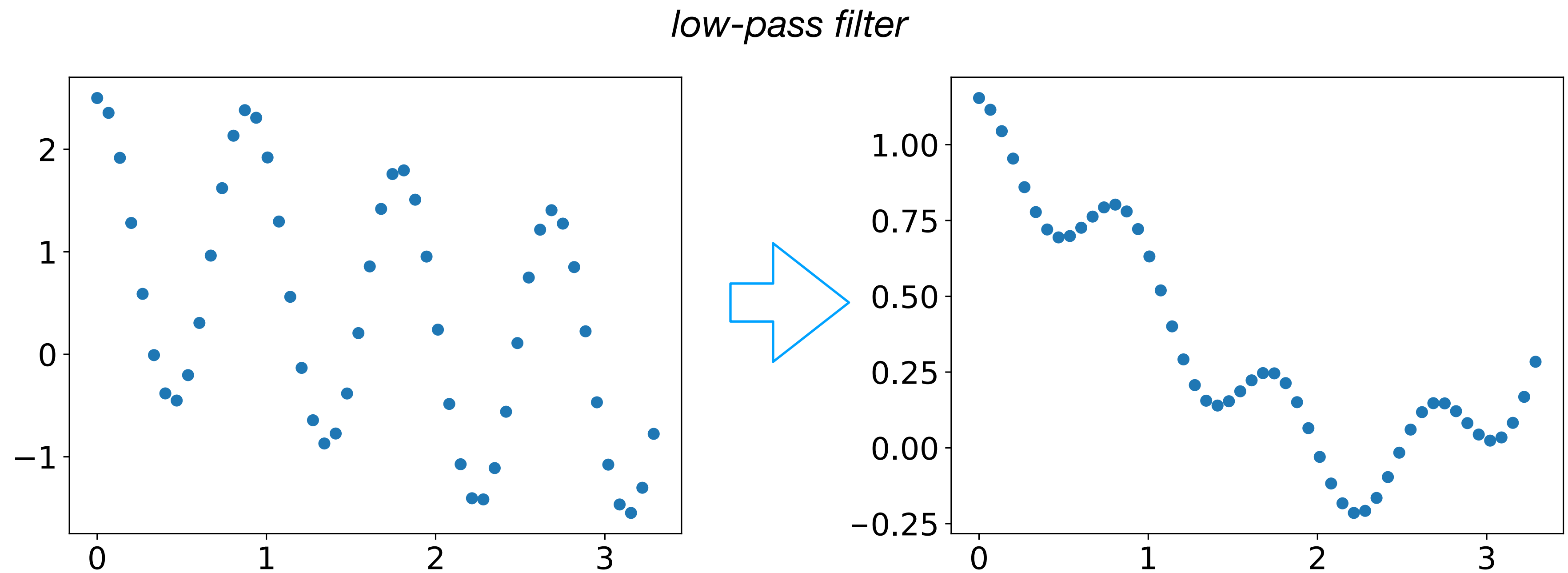


Fewer knobs



image credit: ChatGPT

***Filter =
linear
function***



- Each coordinate of filtered signal is a linear function of input signal: $y = Wx$
 - ▶ $x, y \in \mathbb{R}^{50}$
 - ▶ $W \in \mathbb{R}^{50 \times 50}$

Similar behavior over time

- Audio filters are special matrices: behavior of filter stays fixed over time
 - ▶ i.e., the linear function that gives us y_{23} as a function of x is “the same as” the linear function that gives us y_{42}
 - ▶ “same” means the coefficients are the same but shifted in time

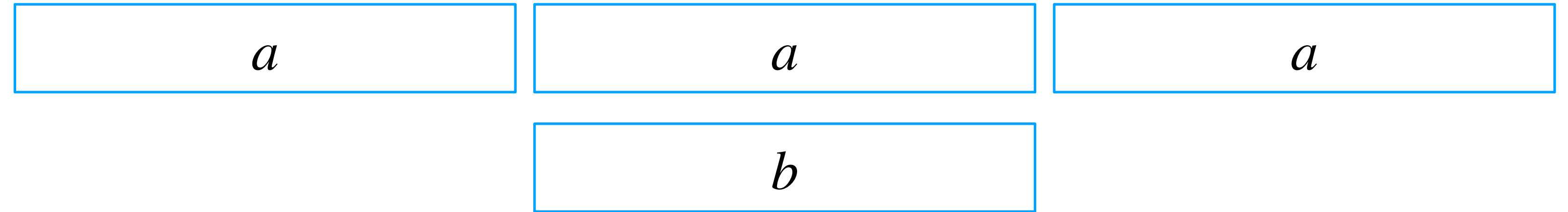
$$\begin{pmatrix} 0.33 & 0.33 & 0.33 & 0 & 0 & 0 & 0 & \dots \\ 0 & 0.33 & 0.33 & 0.33 & 0 & 0 & 0 & \dots \\ 0 & 0 & 0.33 & 0.33 & 0.33 & 0 & 0 & \dots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \ddots \end{pmatrix}$$

- This operation is called *convolution*, and we can learn its weights (the filter) from examples

i.e., each output (say y_{17}) is a target (label); corresponding features are a window of nearby inputs (say $(x_{15}, x_{16}, \dots, x_{19})$)

Convolution

shift b along a , take dot product at each position



$$a = (1 \ 2 \ 3 \ 4 \ 5 \ 4 \ 3 \ 2 \ 1)$$

$$b = (1 \ -1 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0)$$

$$a * b =$$

- Given two vectors $a, b \in \mathbb{R}^d$, their *convolution* is

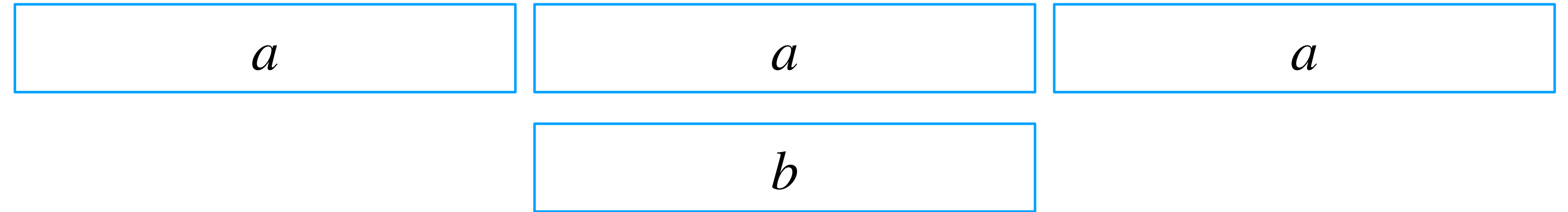
- ▶ $[a * b]_i = \sum_j a_{i+j \bmod d} b_j$

convention: 0-based indexing

note: deep net version

Convolution

shift b along a , take dot product at each position



$$a = (1 \ 2 \ 3 \ 4 \ 5 \ 4 \ 3 \ 2 \ 1)$$

$$b = (1 \ -1 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0)$$

$$a * b = ((1 - 2) = -1$$

- Given two vectors $a, b \in \mathbb{R}^d$, their *convolution* is

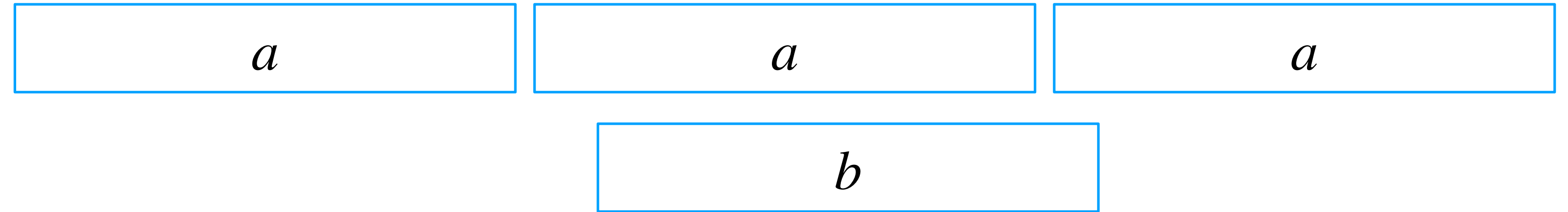
- ▶ $[a * b]_i = \sum_j a_{i+j \bmod d} b_j$

convention: 0-based indexing

note: deep net version

Convolution

shift b along a , take dot product at each position



$$a = (1 \ 2 \ 3 \ 4 \ 5 \ 4 \ 3 \ 2 \ 1)$$

$$b = (1 \ -1 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0)$$

$$a * b = (-1 \ (2 - 3) = -1)$$

- Given two vectors $a, b \in \mathbb{R}^d$, their *convolution* is

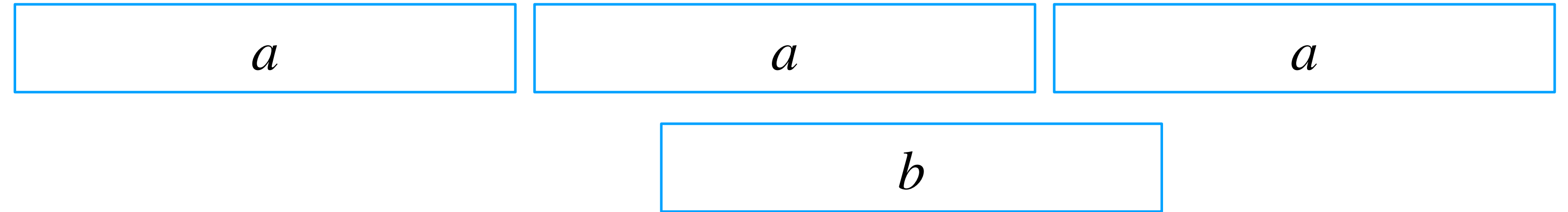
- ▶ $[a * b]_i = \sum_j a_{i+j \bmod d} b_j$

convention: 0-based indexing

note: deep net version

Convolution

shift b along a , take dot product at each position



$$a = (1 \ 2 \ 3 \ 4 \ 5 \ 4 \ 3 \ 2 \ 1)$$

$$b = (1 \ -1 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0)$$

$$a * b = (-1 \ -1 \ (3 - 4) = -1)$$

- Given two vectors $a, b \in \mathbb{R}^d$, their *convolution* is

- ▶ $[a * b]_i = \sum_j a_{i+j \bmod d} b_j$

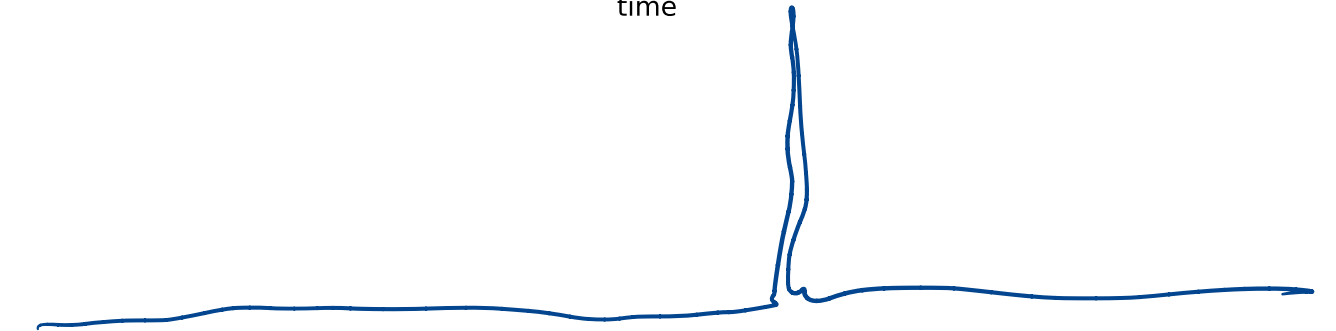
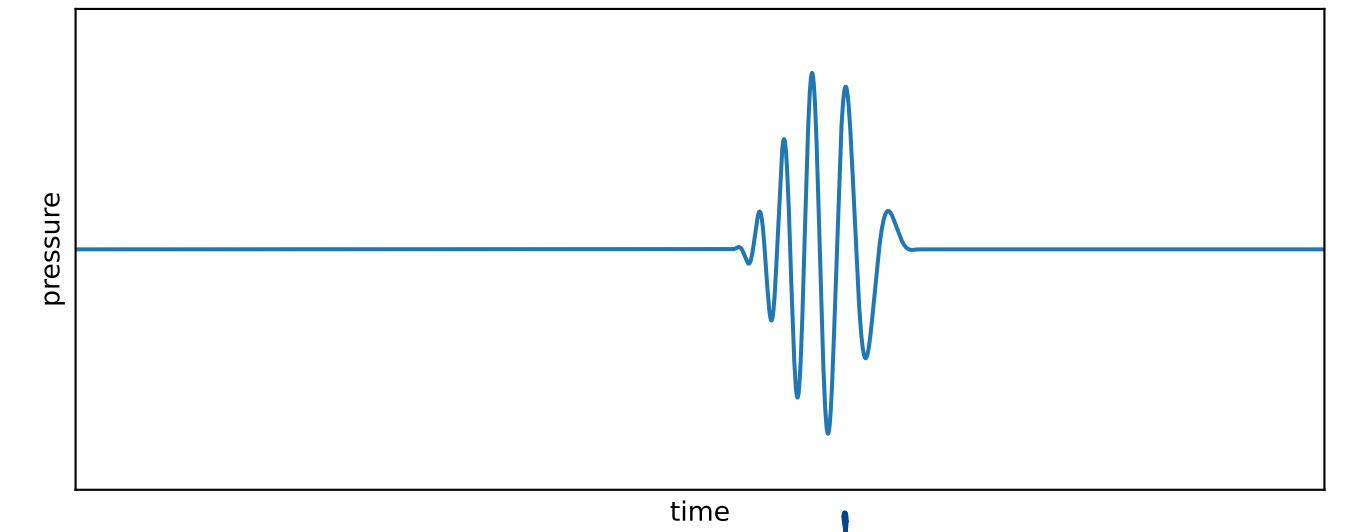
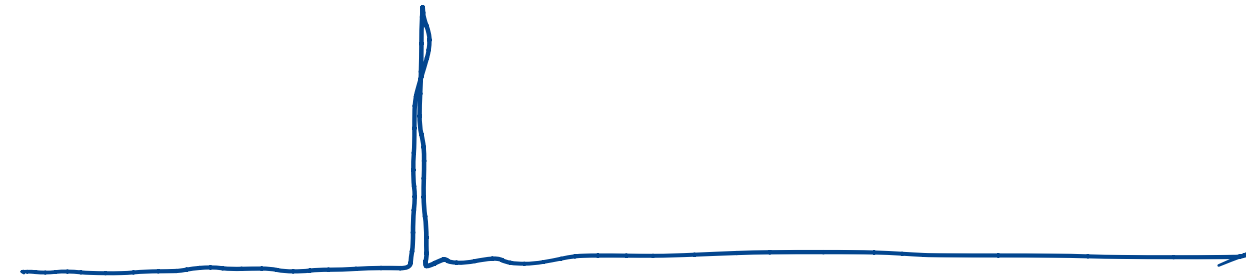
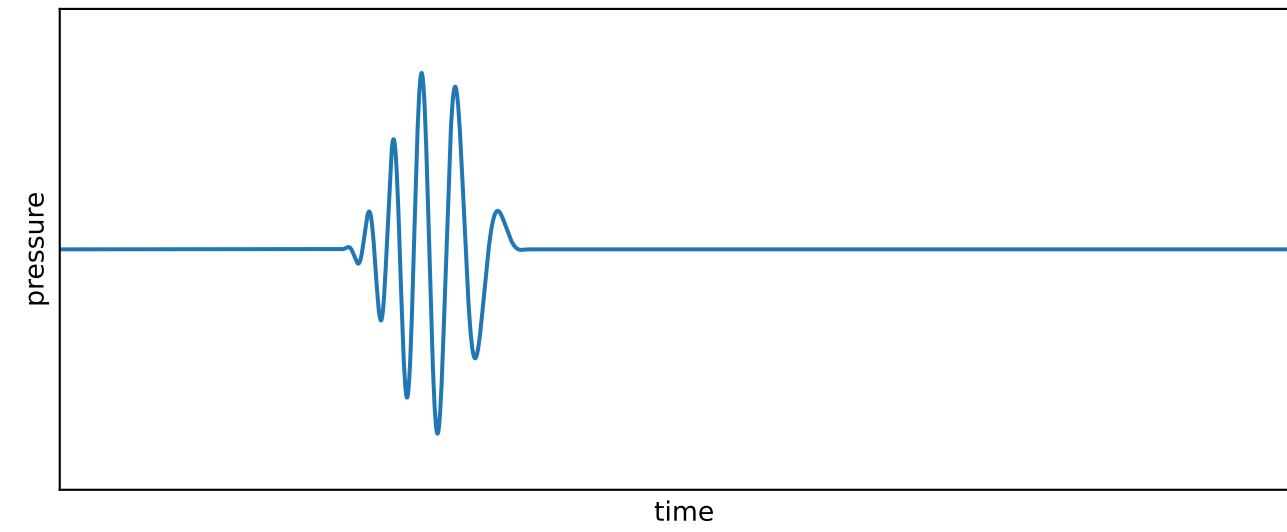
convention: 0-based indexing

note: deep net version

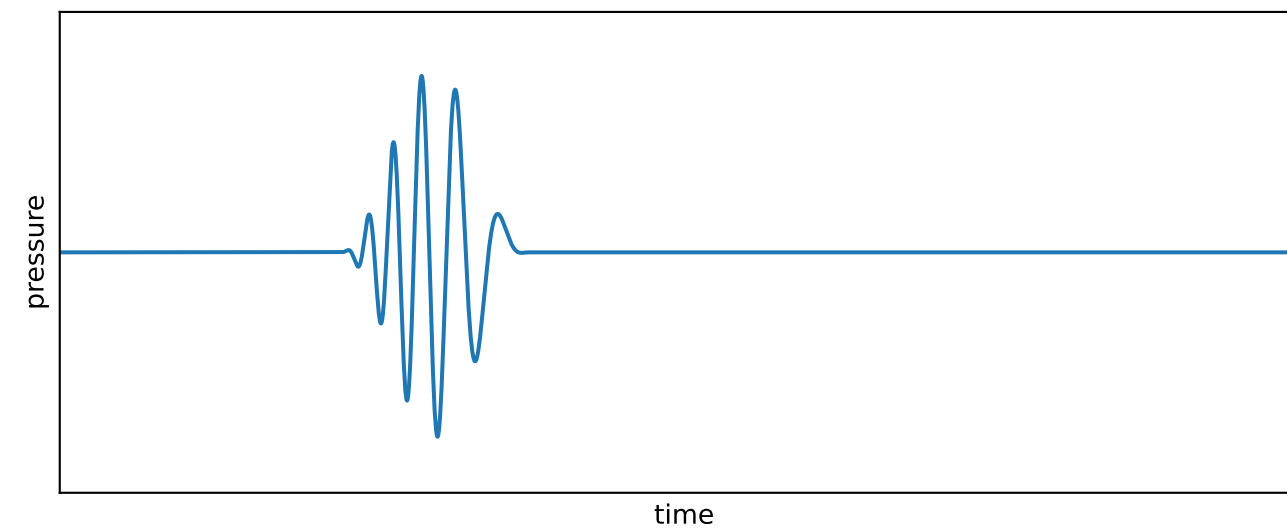
Invariance and equivariance

*to a transformation
of the input, e.g.,
shift it left or right*

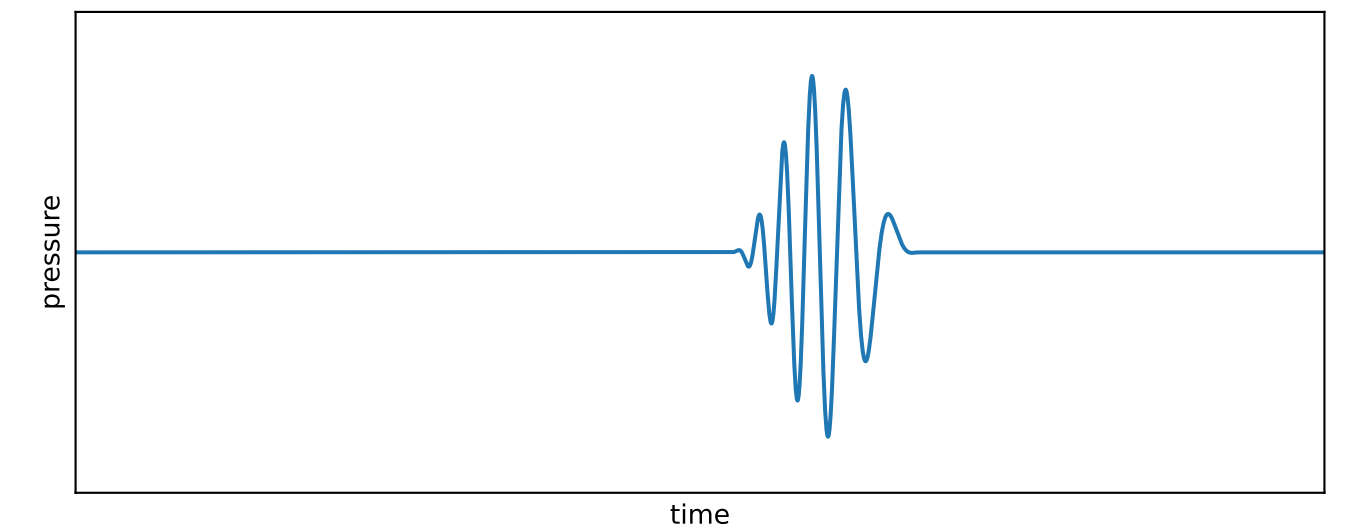
- Equivariance



- Invariance



yes



yes

Convolution is

- Linear
 - ▶ $(a + 2a') * b = a * b + 2(a' * b)$
- Equivariant
 - ▶ shifting $a \rightarrow$ will shift result the same amount $\rightarrow$
 - ▶ shifting $b \rightarrow$ will shift result the same amount $\leftarrow$
- Parsimonious
 - ▶ filter has $O(d)$ parameters (fewer if sparse)
 - ▶ vs. $O(d^2)$ for ordinary linear layer
- Fast, from either
 - ▶ sparsity: $O(ds)$ when b has s nonzeros
 - ▶ FFT: $O(d \ln d)$ via multiplication-convolution theorem

$$a * b = \mathcal{F}^{-1}(\mathcal{F}a \circ \overline{\mathcal{F}b})$$

Variations

Boundary

$a = (1 \ 2 \ 3)$

(1 2 3	1 2 3	1 2 3)	<i>cyclic</i>
(0 0 0	1 2 3	0 0 0)	<i>zero</i>
(3 2 1	1 2 3	3 2 1)	<i>reflect</i>
(1 1 1	1 2 3	3 3 3)	<i>hold</i>

Different size

$a = (1 \ 2 \ 3 \ 4 \ 5 \ 4 \ 3 \ 2 \ 1)$

$b = (1 \ -1)$

→ $b = (1 \ -1 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0)$

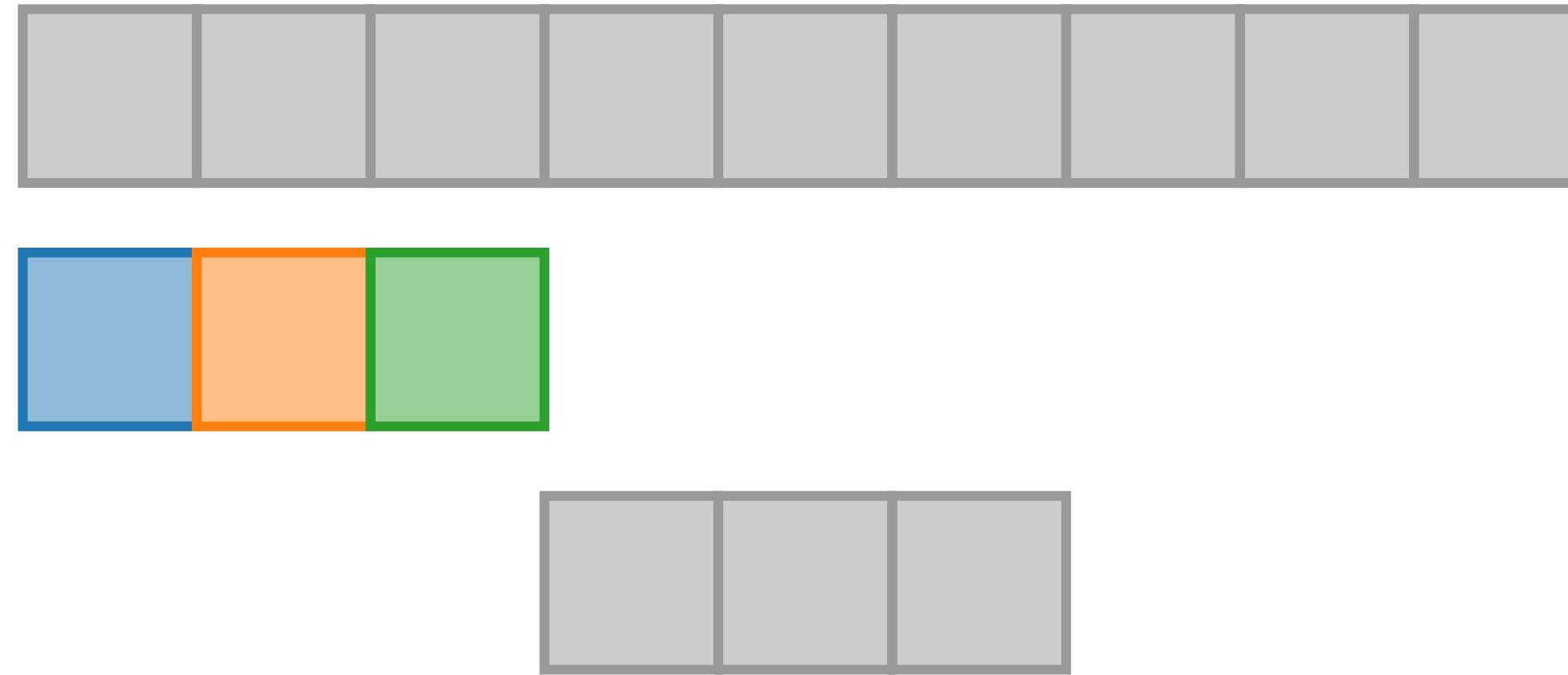
- Boundary conditions
 - ▶ *cyclic* boundary condition (a repeats, as above)
 - ▶ *zero-pad* (add zeros next to a)
 - ▶ *reflect* (add flipped copies of a next to a)
 - ▶ *hold* (repeat closest element of a)
- Different-size vectors: e.g., b smaller than a
 - ▶ handled by boundary conditions — e.g., zero-pad b

$$[a * b]_i = \sum_j a_{i+j} \bmod_d b_j$$

Variations: output size

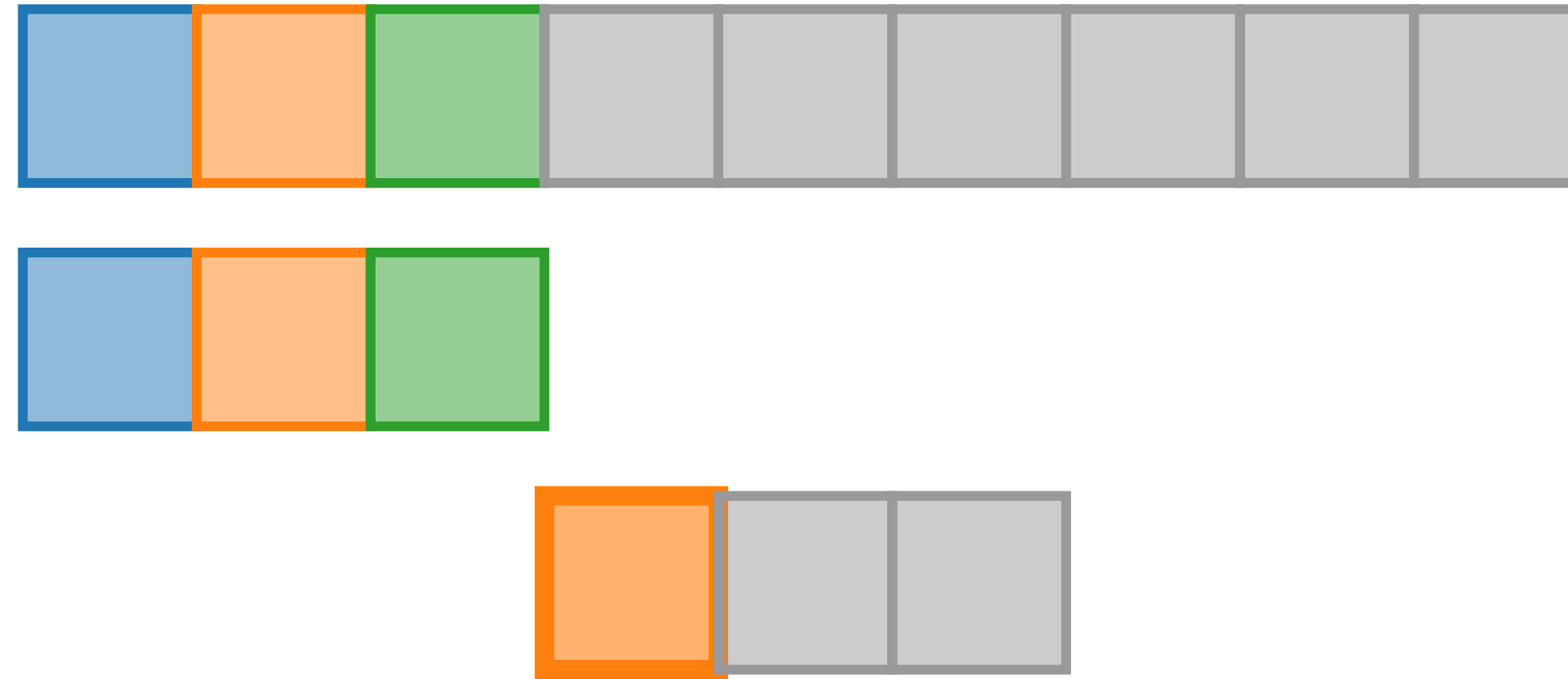
- Size of output
 - ▶ as above: output same size as input
 - ▶ big: keep going as long as b overlaps a
 - ▶ small (“valid”): only create outputs where boundary conditions don’t matter
 - ▶ PyTorch: default small, options control padding/mode

Variations: stride



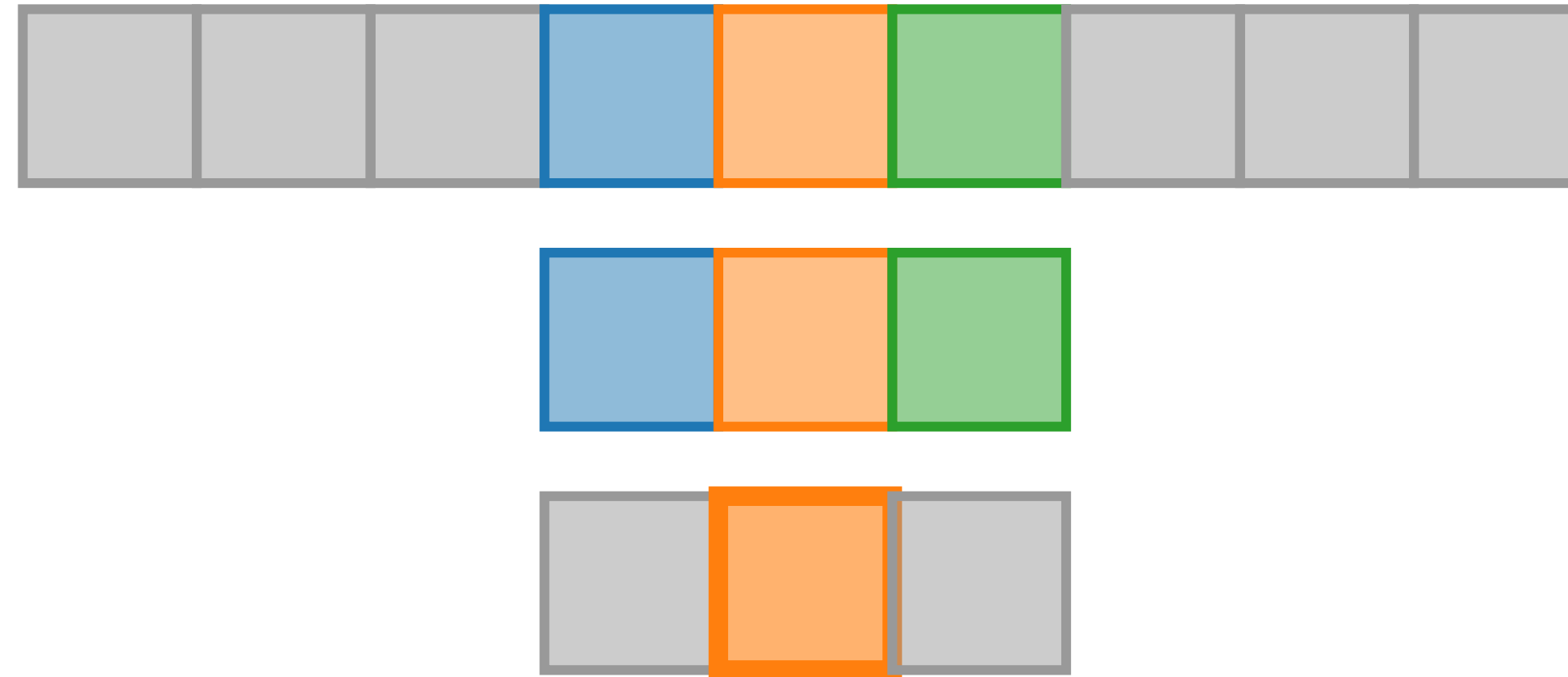
- Stride (downsample to get smaller result)
 - ▶ parameter ℓ tells us to skip starting positions in a
 - ▶ $[a * b]_i = \sum_j a_{\ell i + j \bmod d} b_j$

Variations: stride



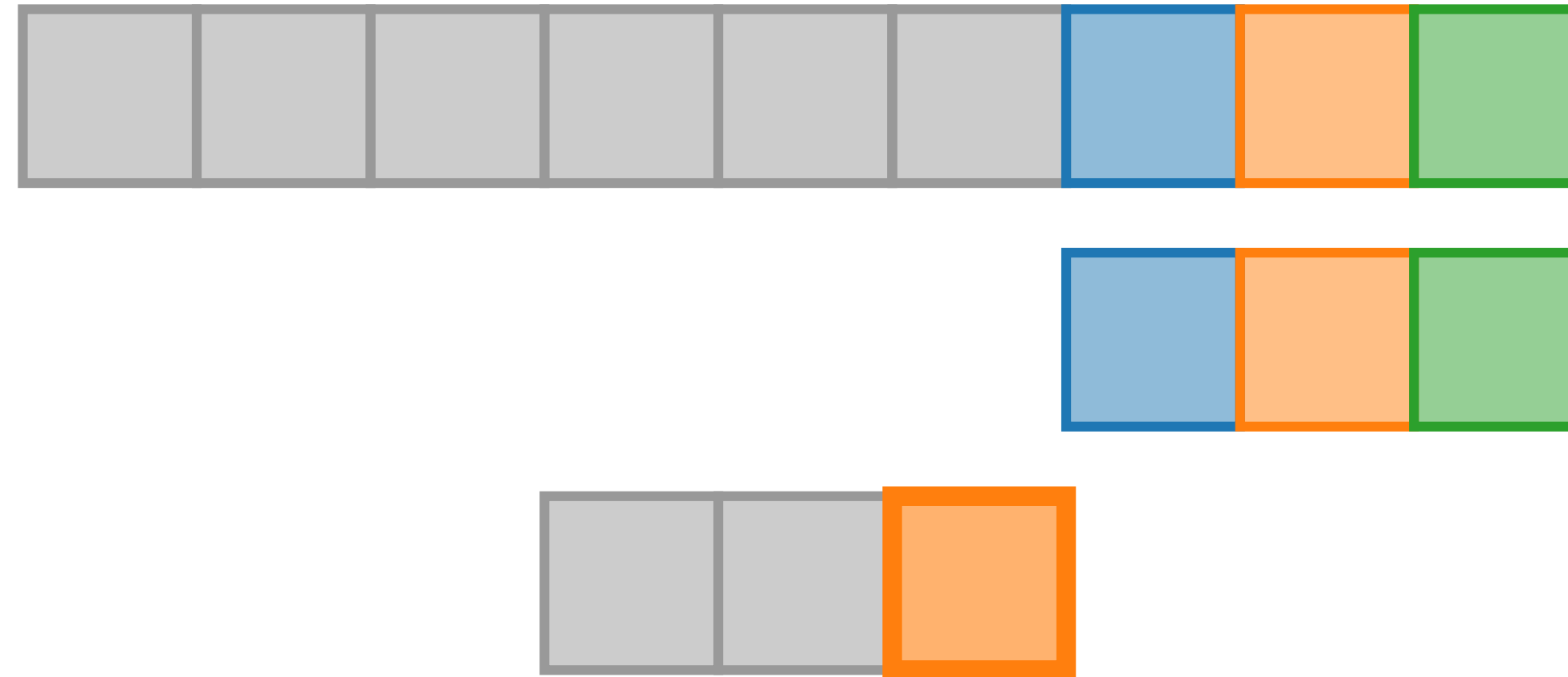
- Stride (downsample to get smaller result)
 - ▶ parameter ℓ tells us to skip starting positions in a
 - ▶ $[a * b]_i = \sum_j a_{\ell i + j \bmod d} b_j$

Variations: stride



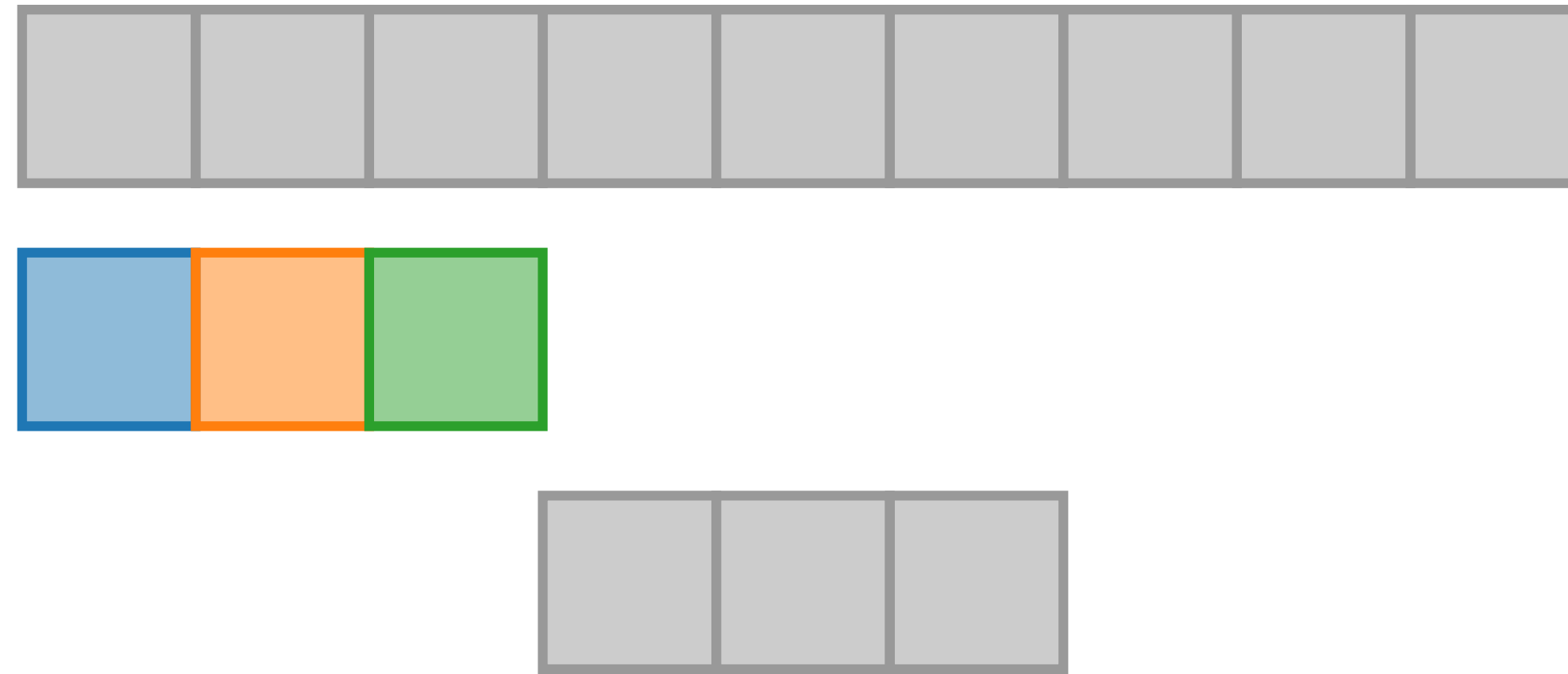
- Stride (downsample to get smaller result)
 - ▶ parameter ℓ tells us to skip starting positions in a
 - ▶ $[a * b]_i = \sum_j a_{\ell i + j \bmod d} b_j$

Variations: stride



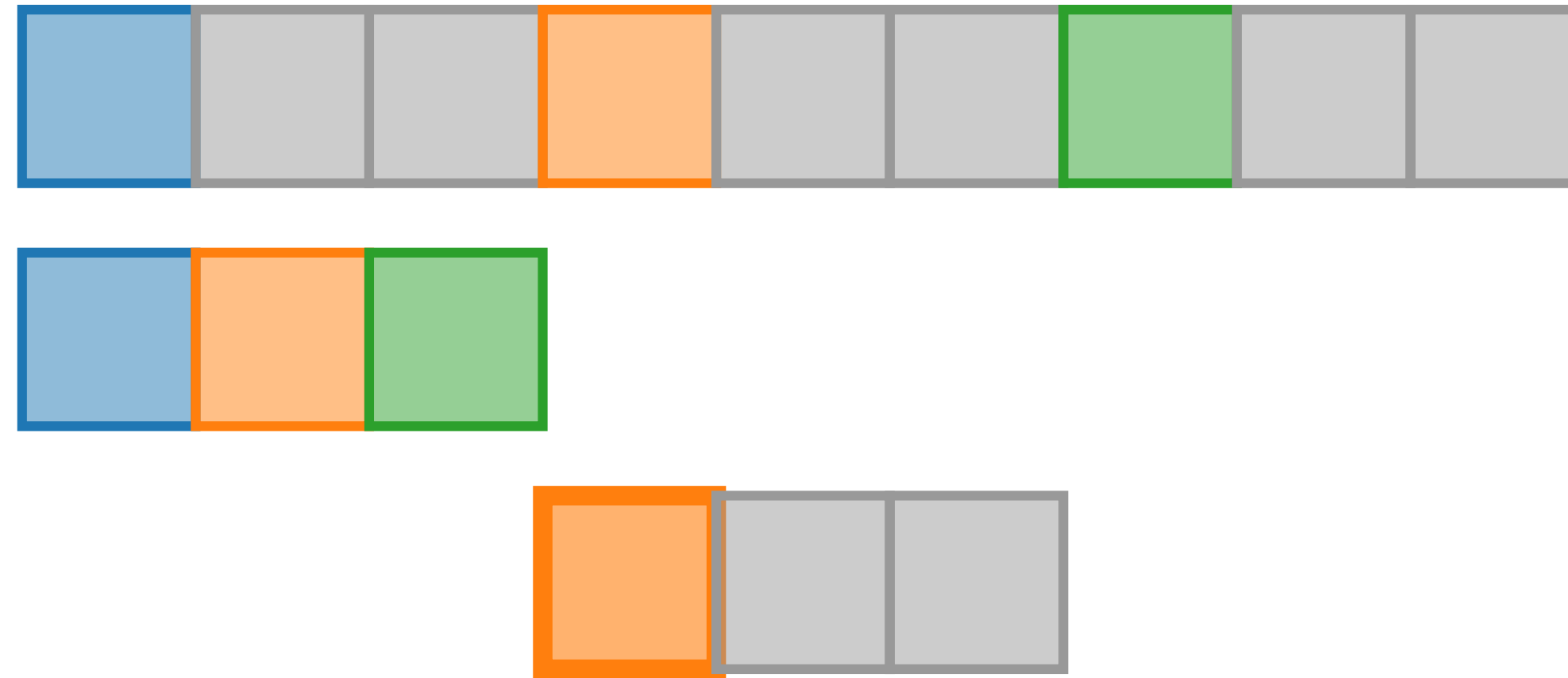
- Stride (downsample to get smaller result)
 - ▶ parameter ℓ tells us to skip starting positions in a
 - ▶ $[a * b]_i = \sum_j a_{\ell i + j \bmod d} b_j$

Variations: dilation



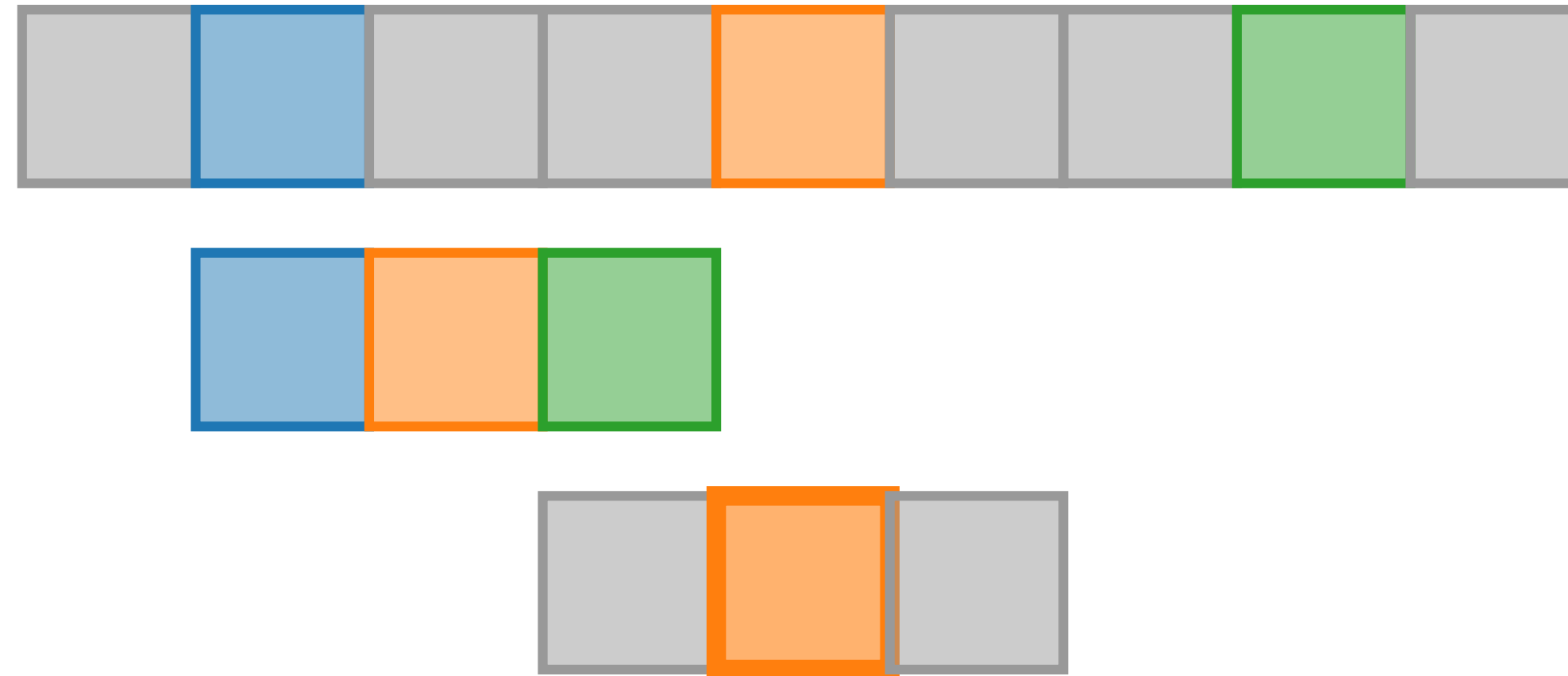
- Dilation (cover larger area with few parameters)
 - ▶ parameter k tells us to spread out b
 - ▶ $[a * b]_i = \sum_j a_{i+kj \bmod d} b_j$

Variations: dilation



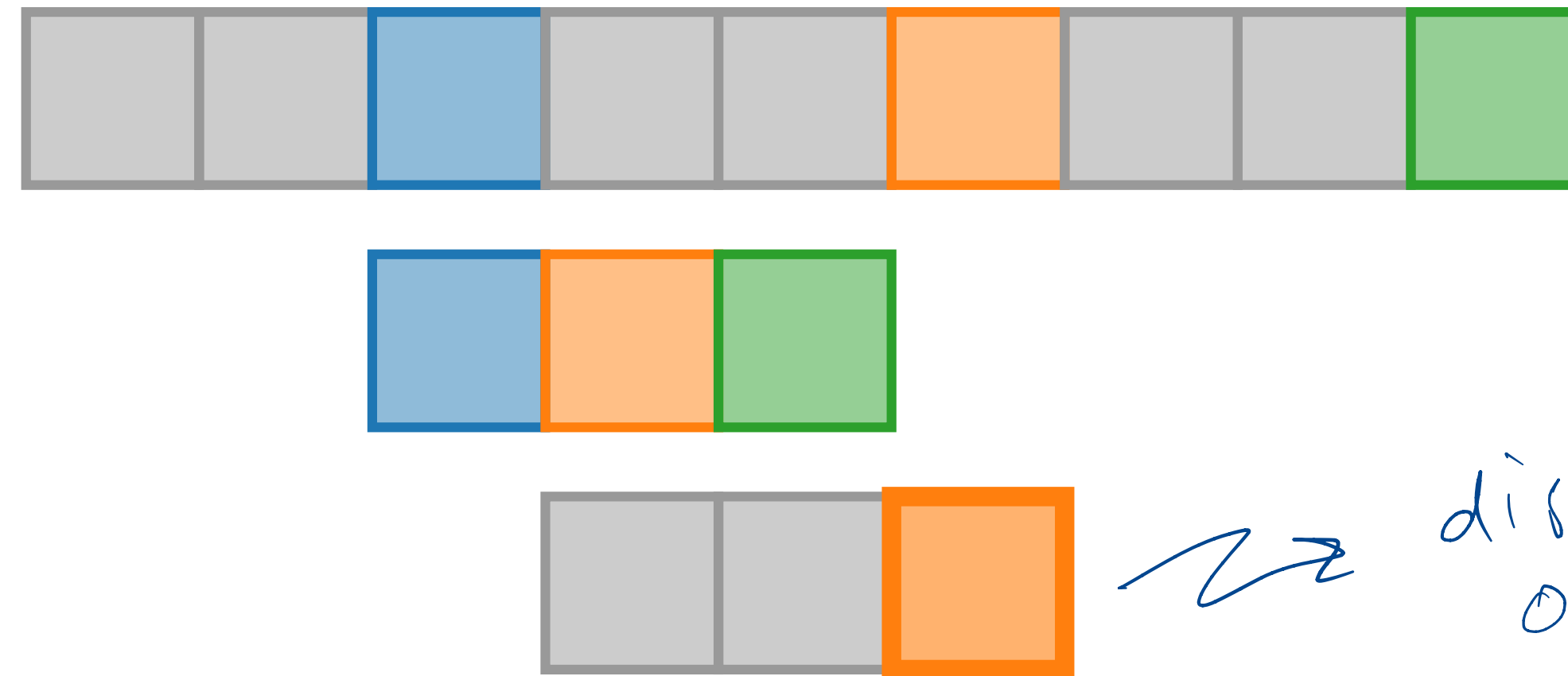
- Dilation (cover larger area with few parameters)
 - ▶ parameter k tells us to spread out b
 - ▶ $[a * b]_i = \sum_j a_{i+kj \bmod d} b_j$

Variations: dilation



- Dilation (cover larger area with few parameters)
 - ▶ parameter k tells us to spread out b
 - ▶ $[a * b]_i = \sum_j a_{i+kj \bmod d} b_j$

Variations: dilation



distinct for
orange filter
element

The fact that output elements are orange is an accident;
they are not aligned with the other orange boxes

- Dilation (cover larger area with few parameters)
 - ▶ parameter k tells us to spread out b
 - ▶ $[a * b]_i = \sum_j a_{i+kj \bmod d} b_j$

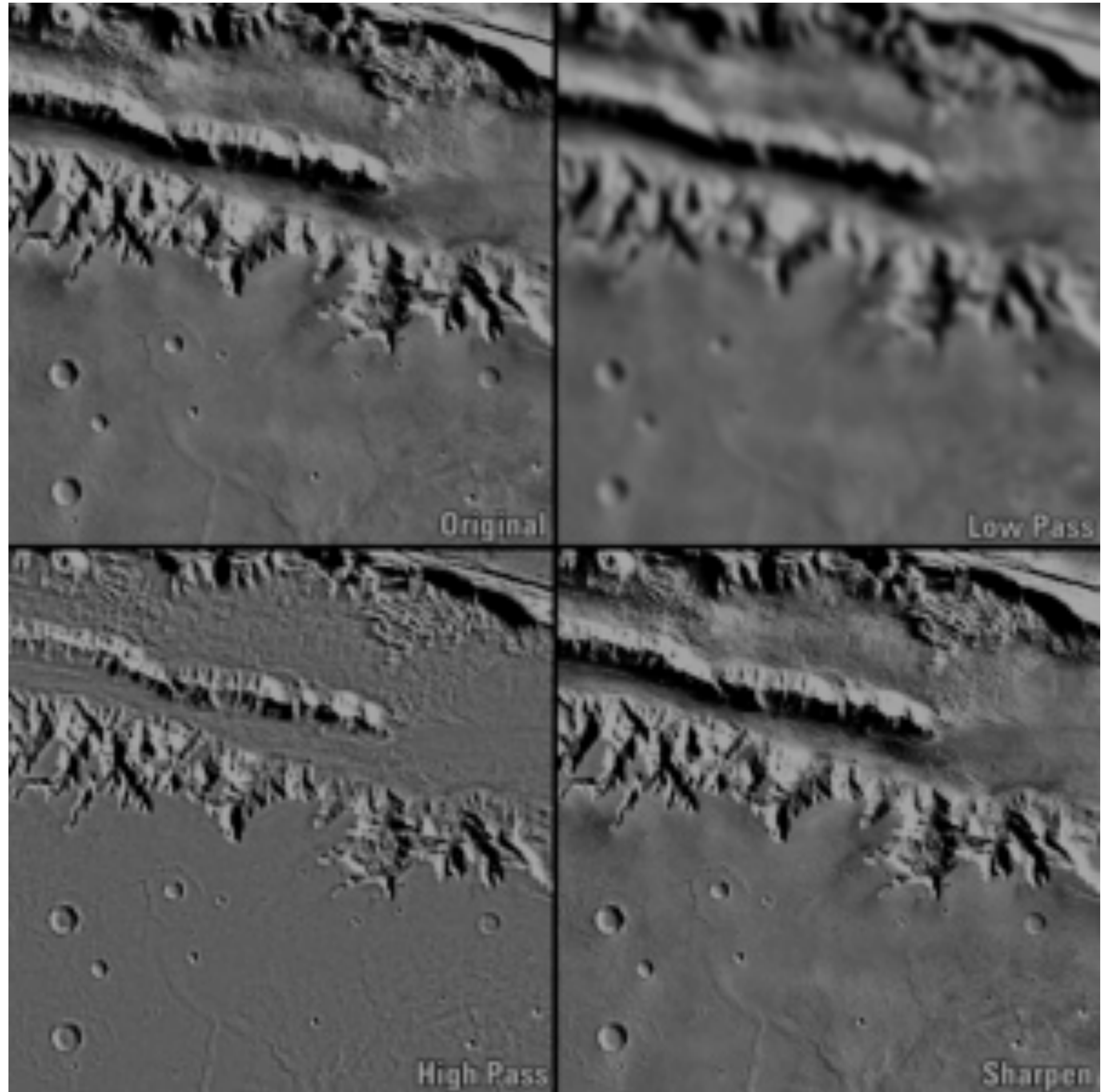
Convolution, invariance, equivariance

- Because convolution is equivariant, it's a great tool for developing equivariant ML models
 - ▶ we'll see some soon
- Also extremely useful for invariant models
 - ▶ e.g., suppose we have an *equivariant* model that outputs a spike when it detects a sound (e.g., a chirp)
 - ▶ if we sum its outputs across time, we get # of chirps
 - ▶ which is *invariant* to shifts

n -d signals

- Beyond having graphs (1d signals) as inputs, common to have 2d, 3d, or higher
 - ▶ e.g., temperature vs. location across US (2d)
 - ▶ e.g., picture of fluffy kitten downloaded from internet (2d if grayscale, 3d if color)
 - ▶ e.g., gas pressure throughout a reaction vessel (3d)
 - ▶ e.g., value function for a control problem where state is the position of a rigid body (6d)

2-d convolution



credit: USGS

2-d convolution

0	0	0	0	0	0
0	1	2	2	1	0
0	2	4	4	2	0
0	1	3	3	1	0
0	1	2	3	1	0
0	0	1	1	0	0

 *

0	1	0
1	-4	1
0	1	0

Just like 1-d convolution, we place the filter at each possible position over the input, then take “dot product”

2-d convolution

0	0	0	0	0	0
0	1	2	2	1	0
0	2	4	4	2	0
0	1	3	3	1	0
0	1	2	3	1	0
0	0	1	1	0	0

 $*$

0	1	0
1	-4	1
0	1	0

 $=$

0			

$$(0 * 0) + (0 * 1) + (0 * 0) + (0 * 1) + (1 * -4) \\ + (2 * 1) + (0 * 0) + (2 * 1) + (4 * 0) = 0$$

Just like 1-d convolution, we place the filter at each possible position over the input, then take “dot product”

2-d convolution

0	0	0	0	0	0
0	1	2	2	1	0
0	2	4	4	2	0
0	1	3	3	1	0
0	1	2	3	1	0
0	0	1	1	0	0

 *

0	1	0
1	-4	1
0	1	0

 =

0	-1		

$$(0 * 0) + (0 * 1) + (0 * 0) + (1 * 1) + (2 * -4) + (2 * 1) + (2 * 0) + (4 * 1) + (4 * 0) = -1$$

Just like 1-d convolution, we place the filter at each possible position over the input, then take “dot product”

2-d convolution

0	0	0	0	0	0
0	1	2	2	1	0
0	2	4	4	2	0
0	1	3	3	1	0
0	1	2	3	1	0
0	0	1	1	0	0

 $*$

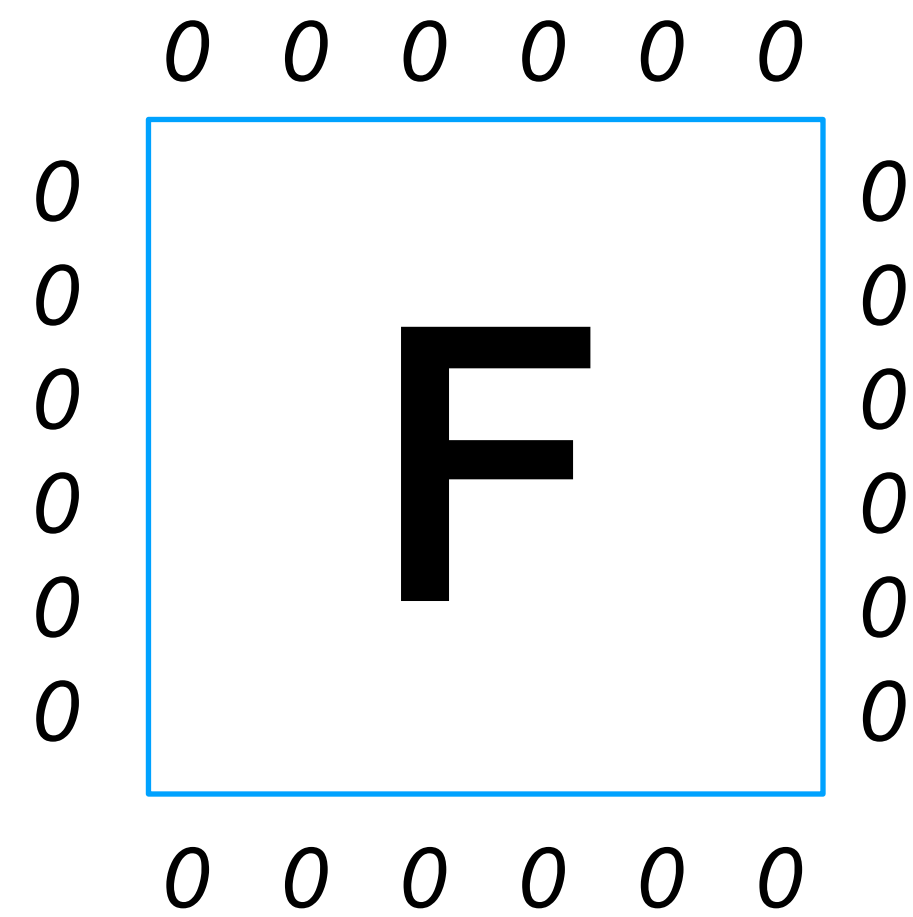
0	1	0
1	-4	1
0	1	0

 $=$

0	-1	-1	0
-2	-5	-5	-2
2	-2	-1	3
-1	0	-5	0

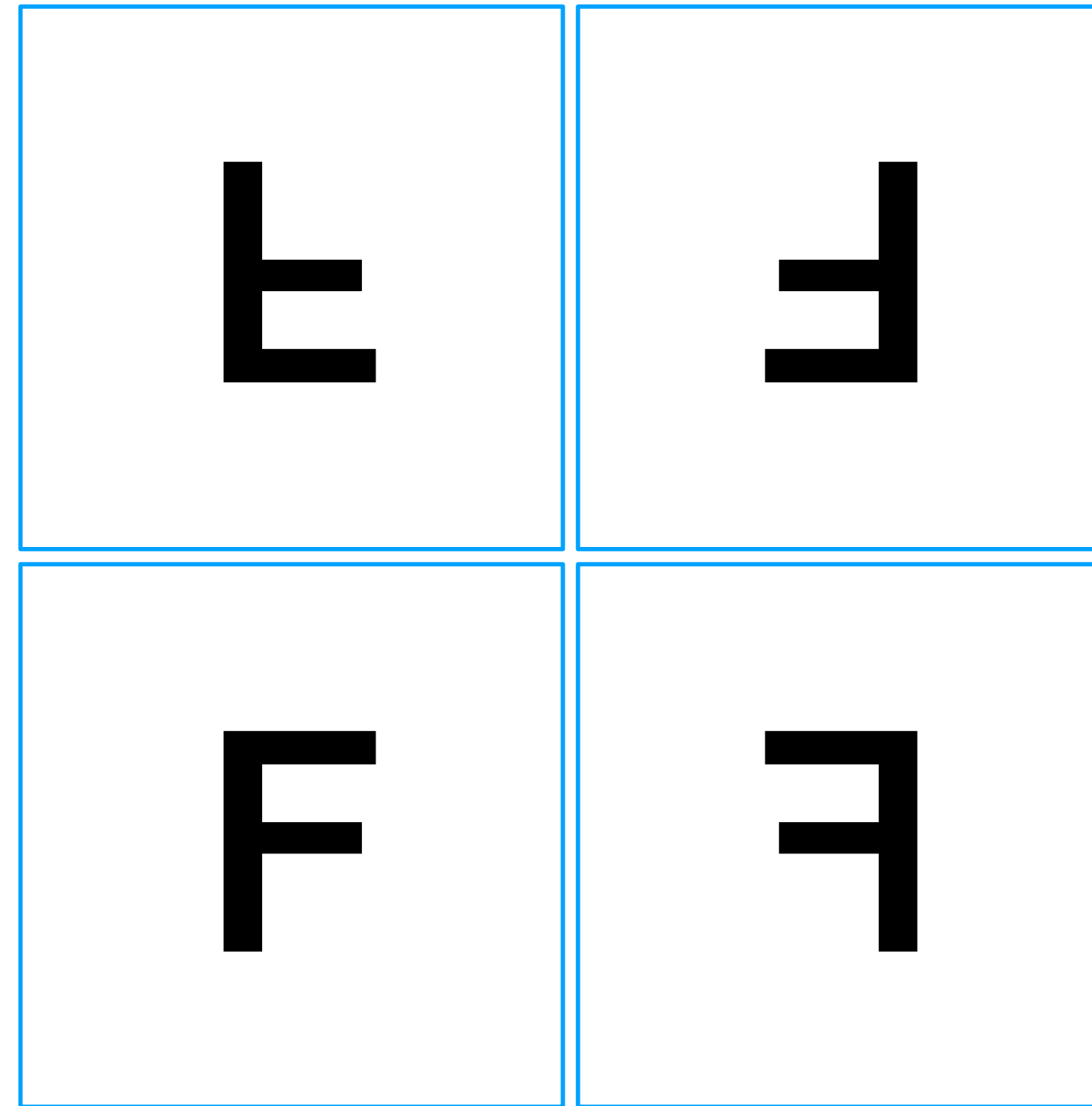
Just like 1-d convolution, we place the filter at each possible position over the input, then take “dot product”

Boundary conditions



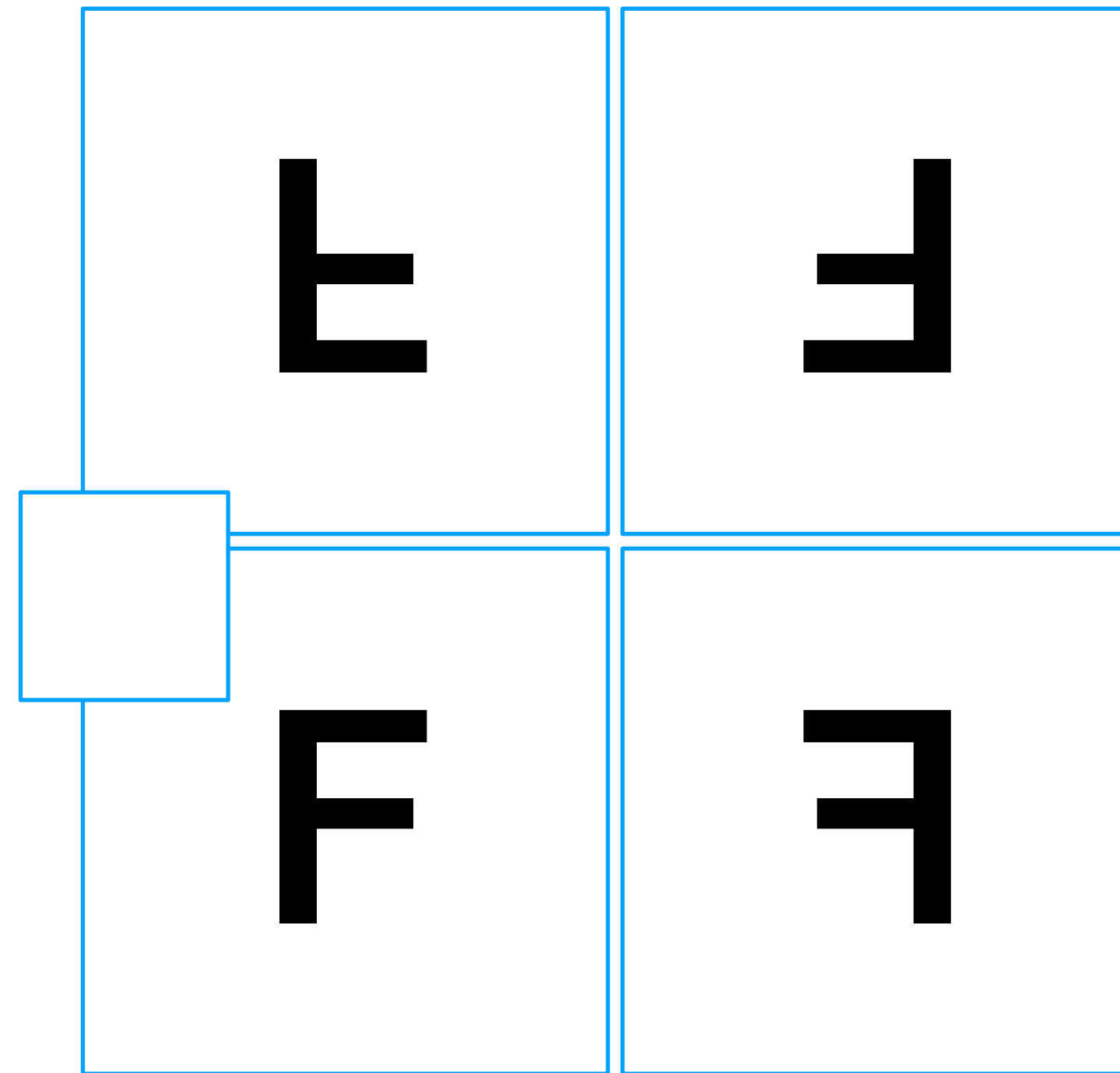
*Similar choices for boundary conditions and size: e.g.,
valid (previous slide), **zero**, or **reflect***

Boundary conditions



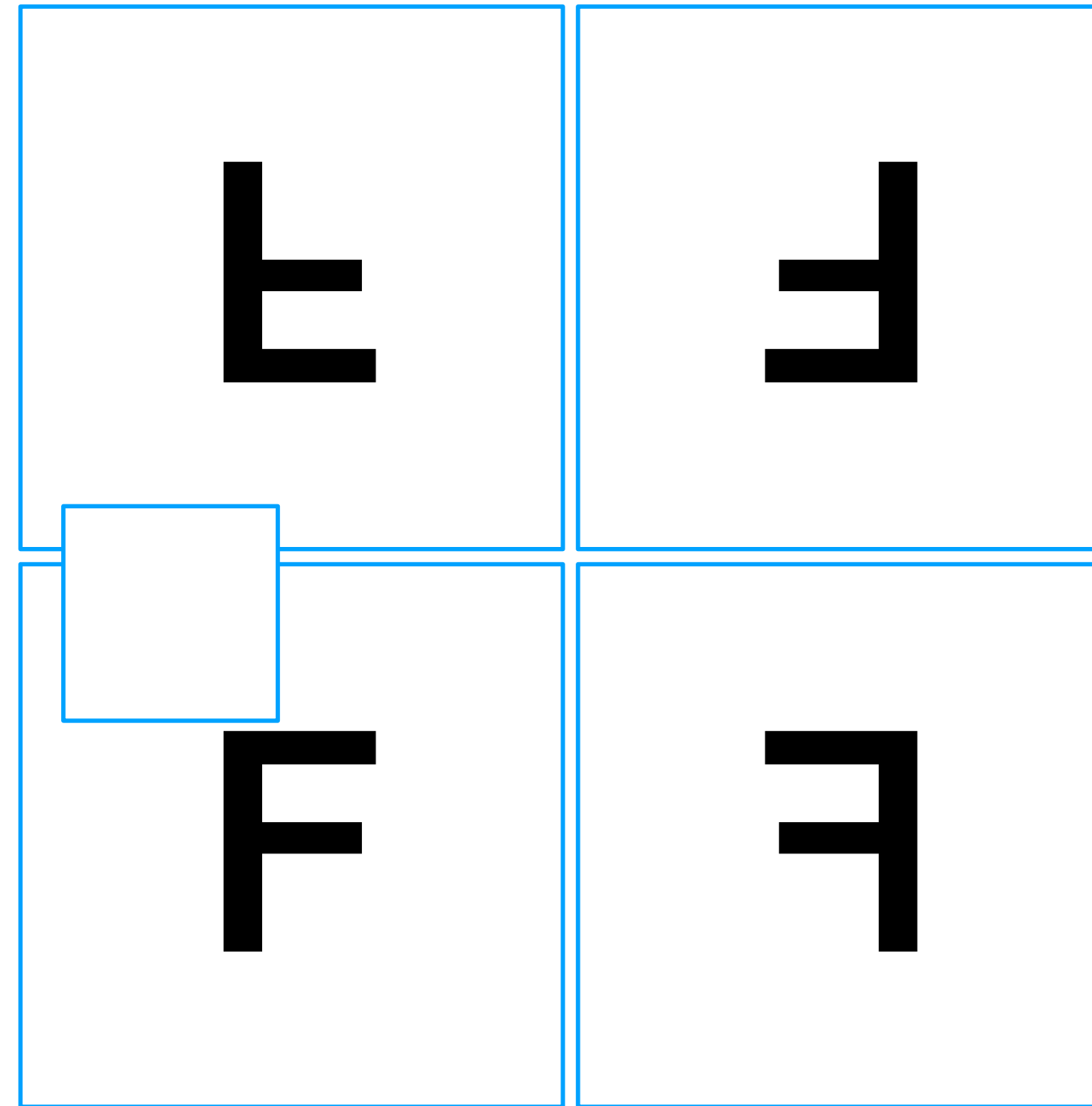
*Similar choices for boundary conditions and size: e.g.,
valid (previous slide), **zero**, or **reflect***

Boundary conditions



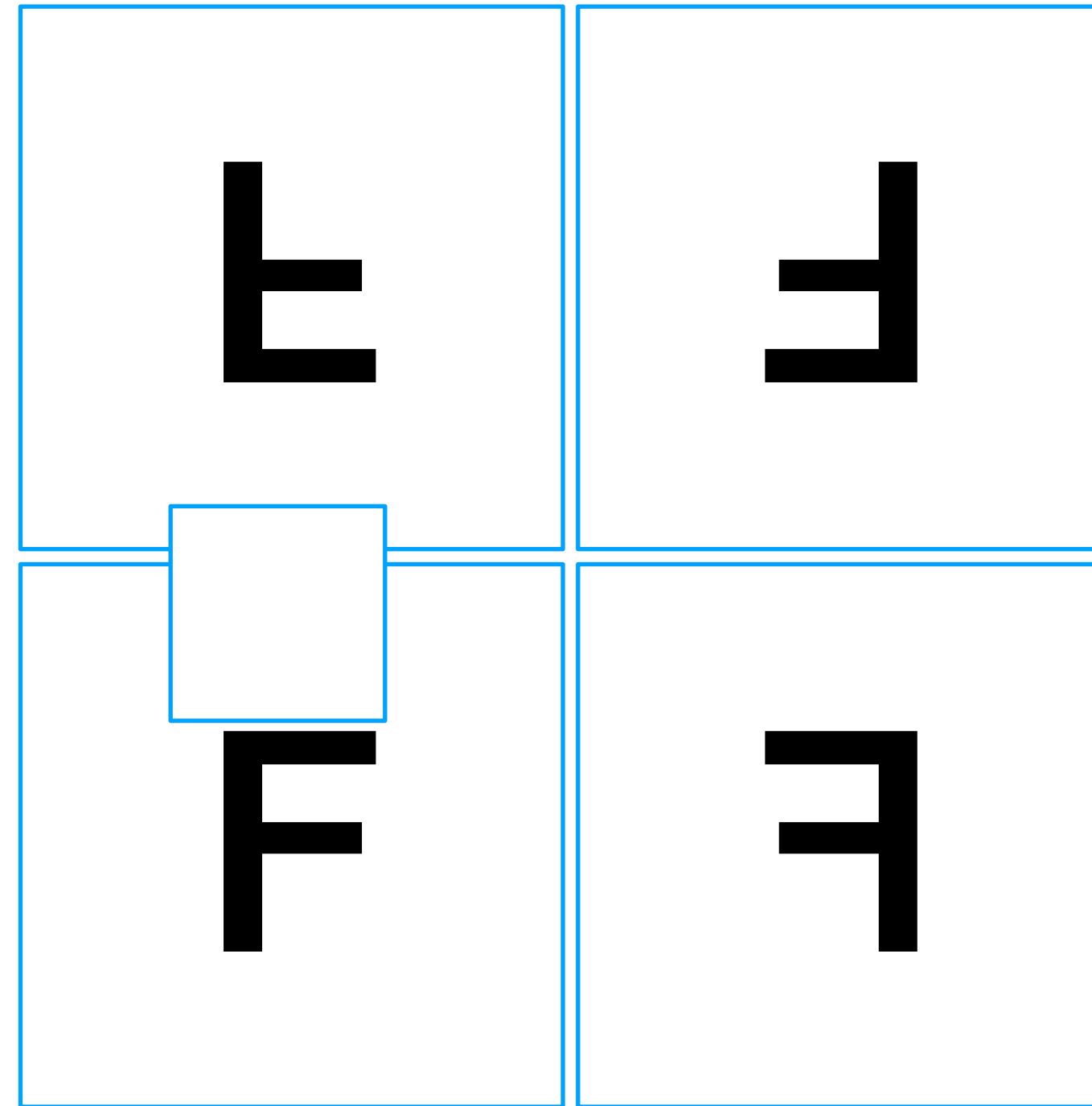
*Similar choices for boundary conditions and size: e.g.,
valid (previous slide), **zero**, or **reflect***

Boundary conditions



*Similar choices for boundary conditions and size: e.g.,
valid (previous slide), **zero**, or **reflect***

Boundary conditions



*Similar choices for boundary conditions and size: e.g.,
valid (previous slide), **zero**, or **reflect***

Equivariance

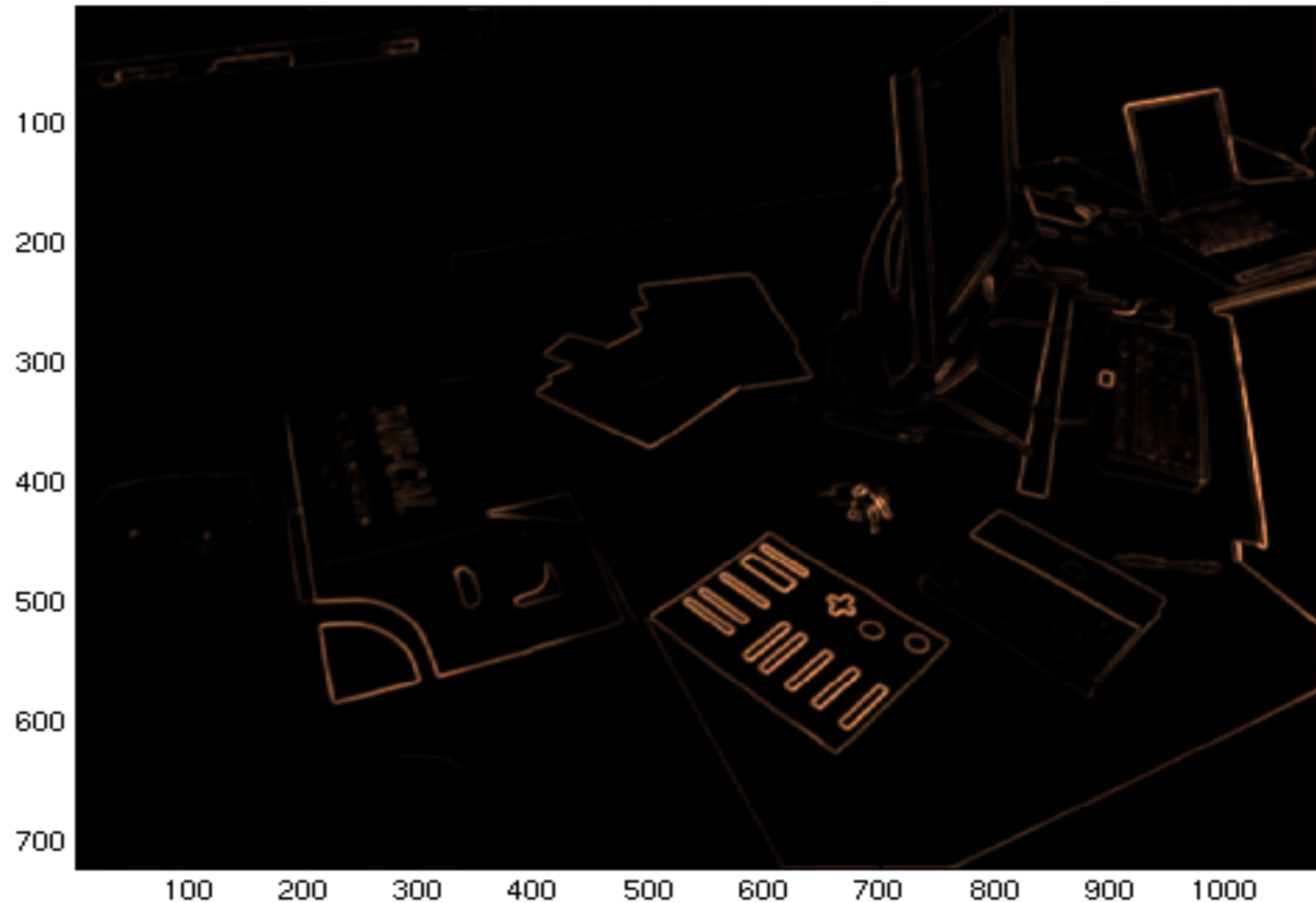
- Convolution remains equivariant in 2-d
 - ▶ in fact, in any number of dimensions
- Shift input any amount in any direction → output shifts same amount in same direction

What can convolution do?



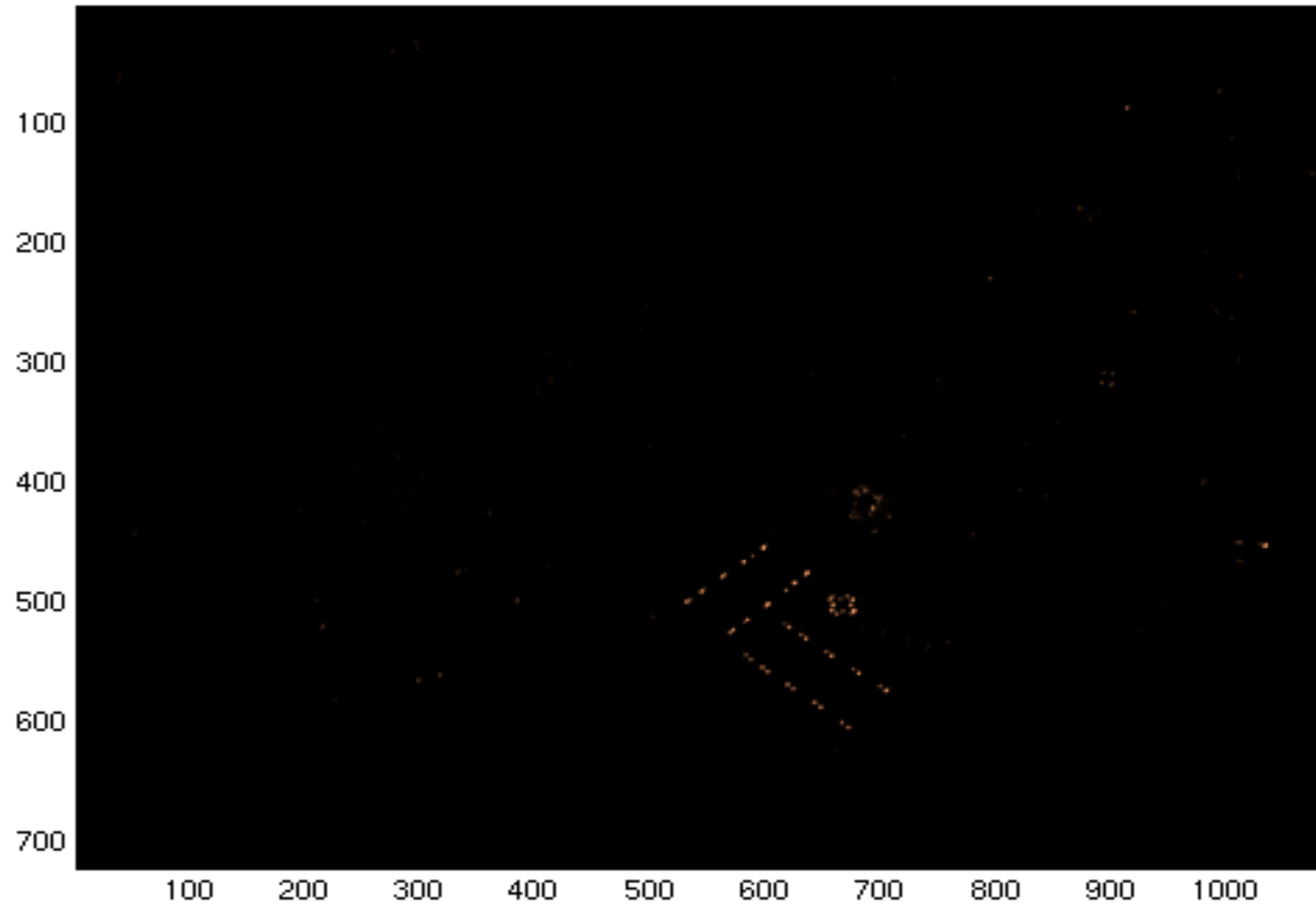
- We've seen: low-pass filter (smoothing), high-pass filter, sharpen (high-pass + residual), edge detection
- Other common uses: detrending, equalization of brightness and contrast, corner detection
 - ▶ here: fixed filter and nonlinearity

What can convolution do?



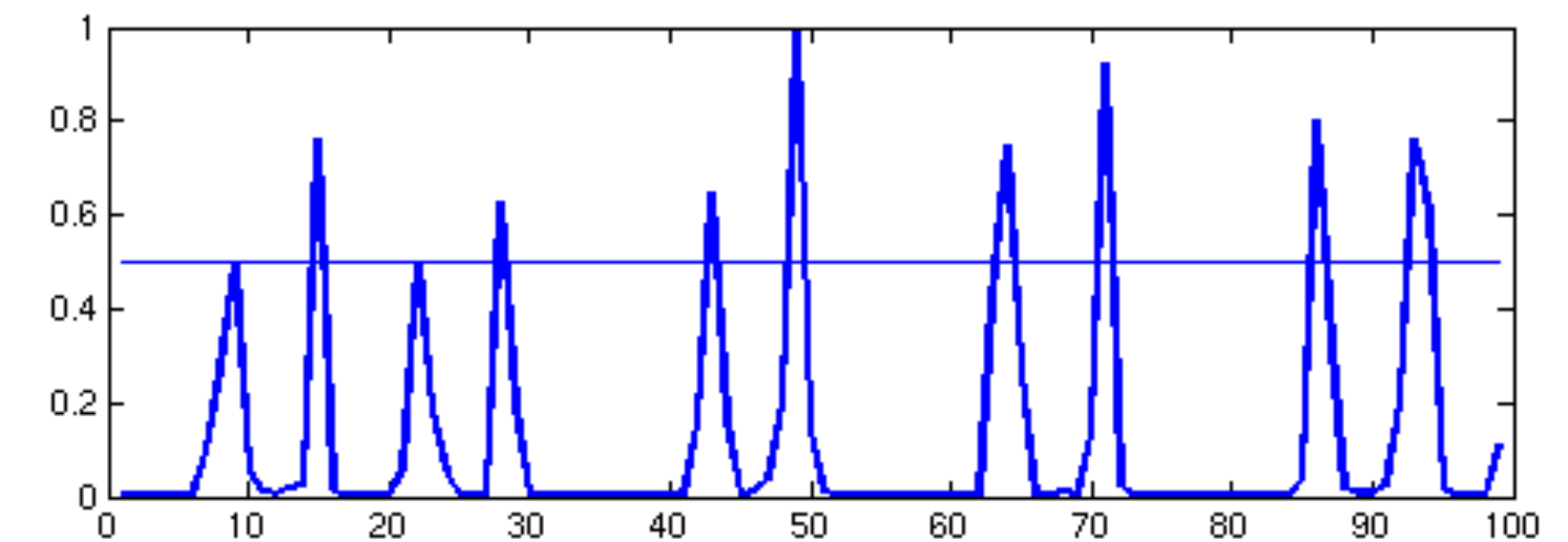
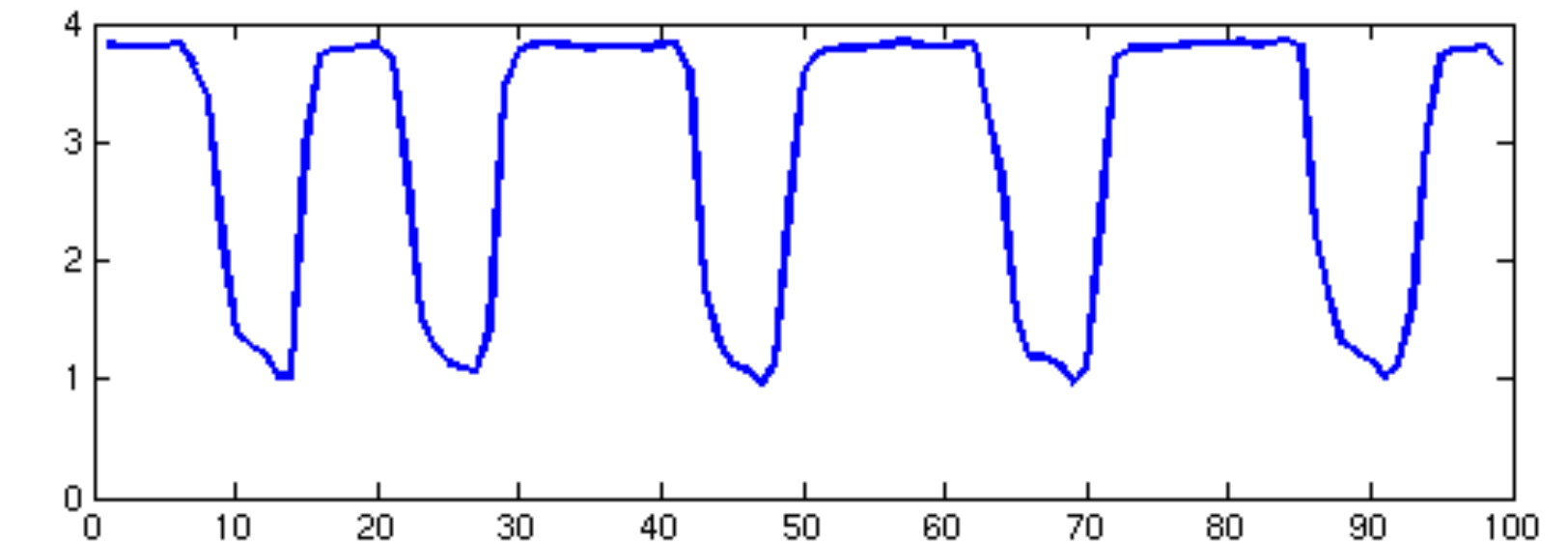
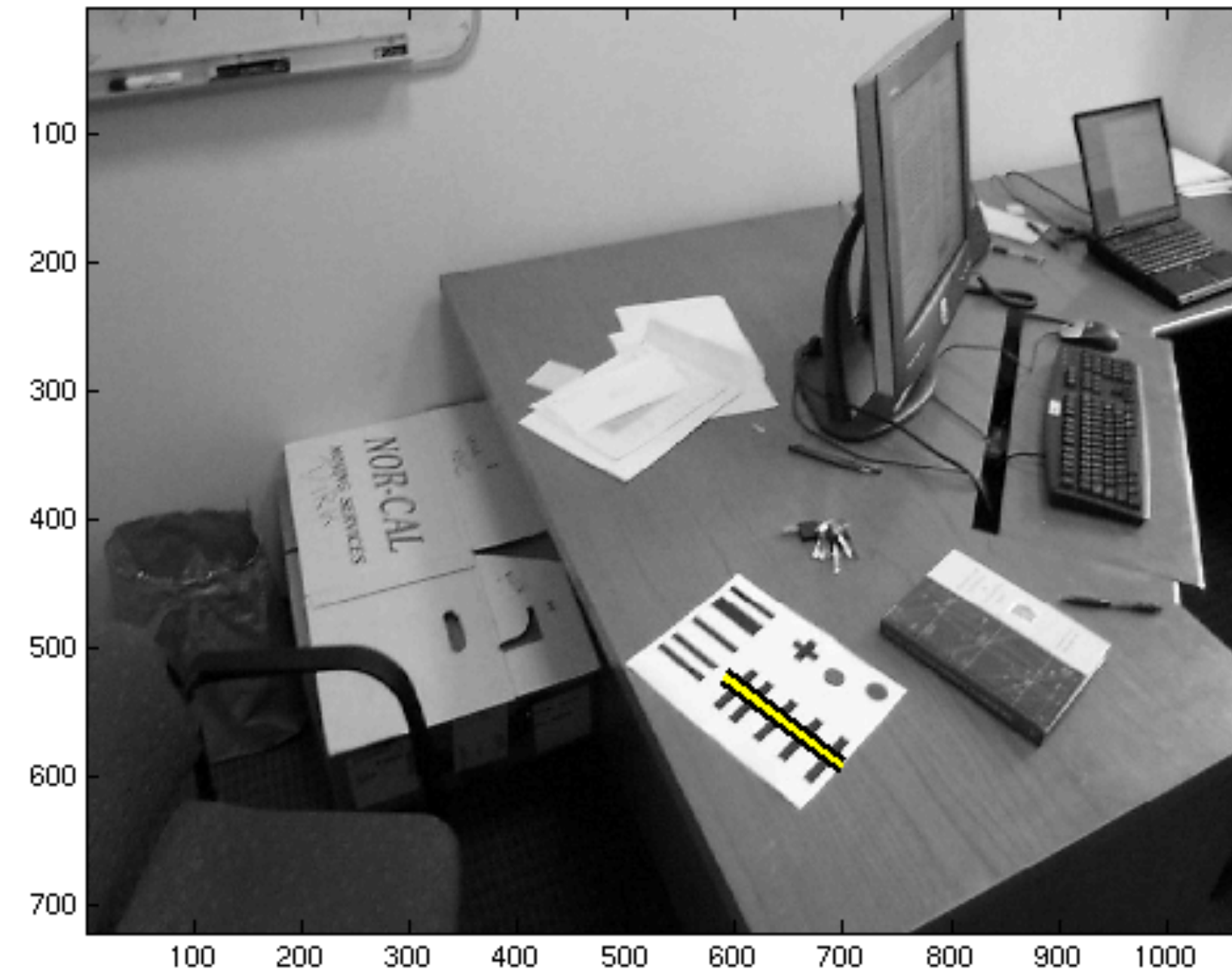
- We've seen: low-pass filter (smoothing), high-pass filter, sharpen (high-pass + residual), edge detection
- Other common uses: detrending, equalization of brightness and contrast, corner detection
 - ▶ here: fixed filter and nonlinearity

What can convolution do?



- We've seen: low-pass filter (smoothing), high-pass filter, sharpen (high-pass + residual), edge detection
- Other common uses: detrending, equalization of brightness and contrast, corner detection
 - ▶ here: fixed filter and nonlinearity

What can convolution do?



- Simple application: detect and read bar codes in an image (what your phone does with QR codes)

Signals as tensors

- An n -dimensional signal is represented as a tensor with n indices
 - ▶ e.g., grayscale image: 2 indices (pixel i, j)
 - ▶ e.g., color image: 3 indices (last is channel: R, G, or B)
 - ▶ e.g., pressure in a volume: 3 indices (voxel i, j, k)
- Each index also called a ***mode***
 - ▶ helps with overloading of the word “dimension”
- List of index ranges: the ***shape*** of the tensor
 - ▶ e.g., RGB image at VGA resolution: (640, 480, 3)
 - ▶ shape (3, 4, 2, 5) tensor has 120 total elements
 - ▶ first index can be 0, 1, 2; second 0, 1, 2, 3; ...

Tensor operations

- Componentwise $+$, $\times$, $\cos$, ...

▶ with *broadcasting*:

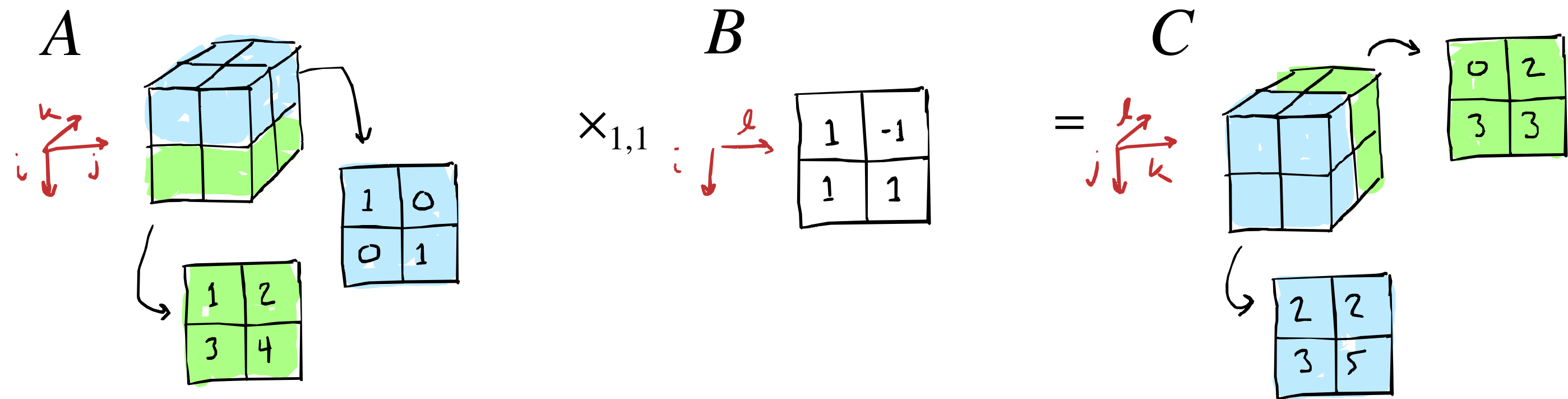
$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} + \begin{pmatrix} 1 \\ 2 \end{pmatrix} = \begin{pmatrix} 2 & 3 & 4 \\ 6 & 7 & 8 \end{pmatrix}$$

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} \times (1 \ 2 \ 3) = \begin{pmatrix} 1 & 4 & 9 \\ 4 & 10 & 18 \end{pmatrix}$$

$$\begin{pmatrix} 1 & 1 & 1 \\ 2 & 2 & 2 \end{pmatrix}$$

- Implemented as NumPy `Array` or PyTorch `Tensor`

Tensor operations



- Contraction: e.g., $A \times_{3,2} B$ or $A \times_{1,1} B$
 - ▶ like matrix multiplication (precise definition below), but using mode 3 of A and mode 2 of B — regular matrix multiplication is $\times_{2,1}$
 - ▶ modes of $A \times_{3,2} B$: modes of A (except 3) then modes of B (except 2)
 - ▶ by convention, all of A 's modes first (in order) then B 's
- Represents a linear function of B with parameters A
 - ▶ or a linear function of A with parameters B